PG DIPLOMA IN AI-ML AND AGENTIC AI ENGINEERING · GUIDED CAPSTONE

SkyWays Logistics Dock coordination under pressure

A two-week guided capstone with three tracks: Machine Learning, Agentic AI, or both

A driver who is going to miss a warehouse appointment sends one short message. Behind that message sit six systems, two dock doors, four other trucks and a coordinator who is already on the phone. You will find the real pain in that moment, go five whys deep, and build one thing that fixes it well enough to demonstrate in ten minutes.

TIME

10 working days, 2 to 3 hours a day, plus a buffer week

OUTPUT

A GitHub repository, a ten-minute demonstration, one proof

STARTER KIT

Database, history, labelled messages, templates, skeletons

Indian Institute of Technology Gandhinagar

Programme delivered with Futurense Technologies · Cohort 1 · Guided Capstone, Supplementary Sitting

Version 1.0

October 2026

How to read this brief

Forty-seven pages, eight parts, one picture per page. The brief is a working document you keep open, not a report you read once.

A · The commitment

How to read this brief 02
The timeline at a glance 03
Three tracks and how to choose 04
What cleared means 05

B · The world

SkyWays in one picture 06
Follow one truck 07
Who hurts, and how 08
Where the truth lives 09
The data model 10
One message, traced 11

C · Think before you build

Why the same problem again fails 12
Five Whys, worked 13
What to do at each why 14
Pain-point depth matrix 15
From root cause to candidates 16
Research moves and prompts 17

D · Scope

PRD-lite in one page 18
Scope ladder: simple to enterprise 19
Guardrails 20

E · Design

Context diagram 21
Containers: Agentic 22
Containers: ML 23
Containers: Hybrid 24
Inside the agent 25
Inside the ML pipeline 26
Sequence: delay to held slot 27
Sequence: two drivers, one slot 28
The commitment state machine 29
Tool contracts 30
Data and feature contracts 31
Prompt collection 32
Evaluation design 33

F · Build

Two weeks on one page 34
Day by day 35
The starter kit 36
Mistakes we expect 37

G · Present

Ten minutes, eight slides 38
The demo script 39
Your repository 40
LinkedIn and resume 41
Facing the panel 42
How you are assessed 43

H · Grow

Room to innovate 44
Datasets to augment 45
Papers, docs and videos 46
Tools and credits 47

HOW EVERY PAGE WORKS

The heading names the topic. The italic line under it states the point. The picture carries the content and the legend decodes it. A confidence line at the foot says whether a device is established practice, contested, or this brief's own construction.

READING ORDER

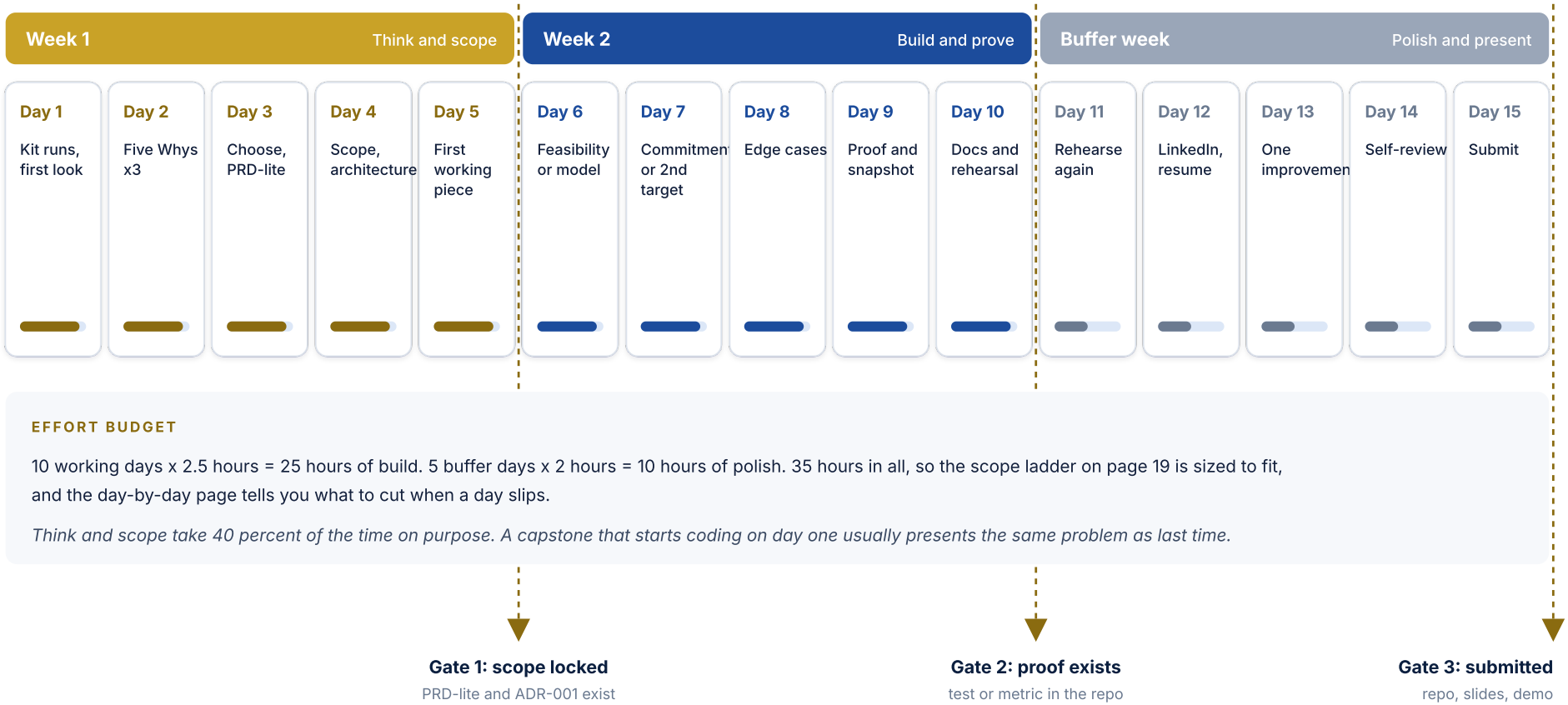
Parts A to D in the first three days, before any code. Part E on day four. Part F open beside you every day. Part G in the last three days. Part H whenever you are stuck or curious.

TRACK MARKERS

A chip at the top right of a page says which tracks it applies to. No chip means every track.

The timeline at a glance

Fifteen working days: five to think, five to build, five to polish. Three gates decide whether you move on.



EFFORT BUDGET

10 working days x 2.5 hours = 25 hours of build. 5 buffer days x 2 hours = 10 hours of polish. 35 hours in all, so the scope ladder on page 19 is sized to fit, and the day-by-day page tells you what to cut when a day slips.

Think and scope take 40 percent of the time on purpose. A capstone that starts coding on day one usually presents the same problem as last time.

LEGEND Week 1, think and scope · Week 2, build and prove · Buffer week · Gate

CONFIDENCE This brief's own construction: the three-gate split is this brief's construction; the hours are a budget, not a measurement

Three tracks and how to choose

Each track proves one thing well. The rubric rewards depth on your track, never breadth across all three.

Machine Learning

Predict what the planner cannot see yet

- Target: unload duration, late arrival, or no-show
- Data: 6,040 historical dock visits
- Core skill proved: honest evaluation on a time split
- Demo: a prediction, its drivers, the planner's action

Pick this if you are faster with pandas and scikit-learn than with LLM tooling, and you want to prove rigour.

Agentic AI

Turn a driver message into a held, feasible slot

- Target: a tool-calling LangGraph agent over SQLite
- Data: 21 live shipments, 148 labelled messages
- Core skill proved: correctness under concurrency
- Demo: a conversation, a hold, a refusal

Pick this if you enjoyed the agent weeks and can write a tool with a typed contract without help.

Hybrid

The agent asks the model before it offers a slot

- Target: Agentic Simple plus one prediction as a tool
- Data: both files, joined by the integration contract
- Core skill proved: two components, one contract
- Demo: a slot sized by a prediction

Pick this only if you can finish Agentic Simple by day 6. Otherwise it becomes two half projects.

DECISION RULE

Answer two questions. Can you, today, write a SQL join and a scikit-learn pipeline from memory? Can you, today, write a function an LLM calls and parse its JSON reply? Yes to the first only: ML. Yes to the second only: Agentic. Yes to both and you finished the agent weeks comfortably: Hybrid. Yes to neither: Agentic Simple, because the kit carries more of it for you. Record the choice in ADR-001 on day four and do not reopen it.

LEGEND Machine Learning track · Agentic AI track · Hybrid track

CONFIDENCE This brief's own construction: the decision rule is this brief's construction

What cleared means

Three pillars, one binary outcome. The assessment page in Part G shows how marks map to these pillars.

Repository

- Runs from a fresh clone with the README steps
- Five Whys, PRD-lite and at least one ADR in docs/
- Architecture picture at the top of the README
- The proof: a passing test or a metric table with a baseline

Demonstration

- Ten minutes, eight slides, timer visible
- Three-minute live demo with a screenshot fallback
- One refusal or escalation shown on purpose
- Every number traceable to a file

Defence

- You explain the root cause without notes
- You name what the system never decides
- You name three limits and their enterprise fix
- You answer the concurrency or leakage question

Cleared

All three pillars stand, on your chosen track, at Simple level or above. The diploma capstone component is recorded as cleared. Nothing in your public artefacts says anything other than capstone.

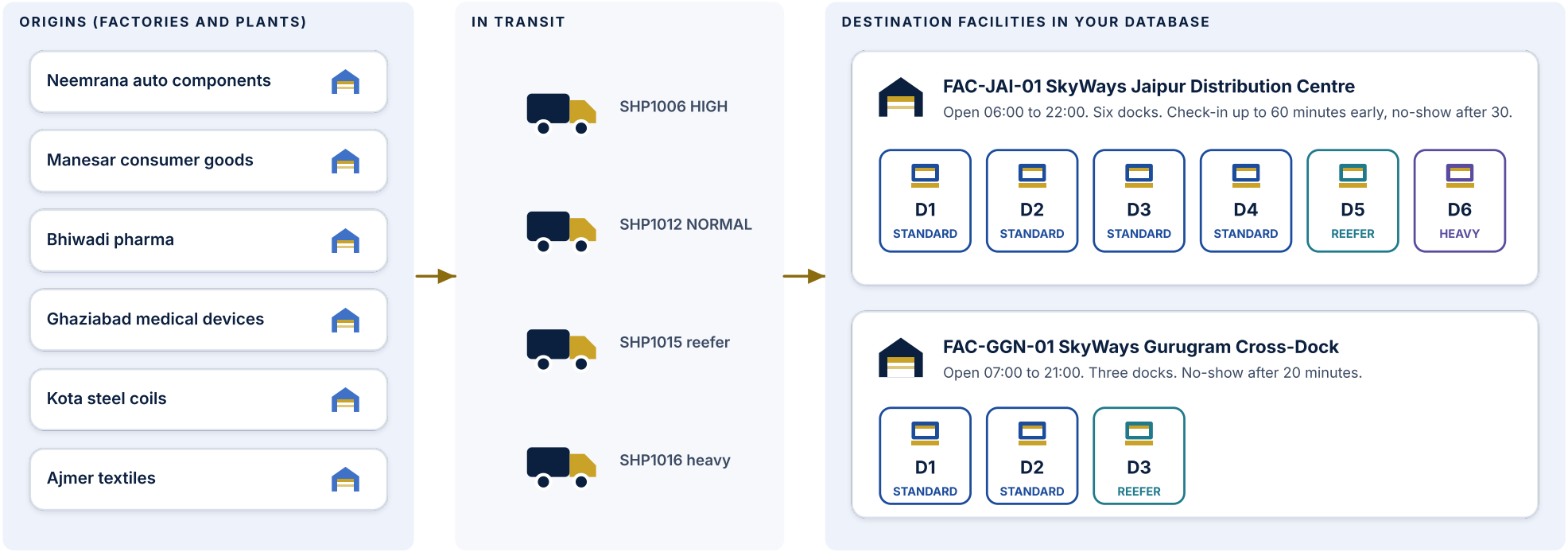
Not cleared

A missing pillar, a demonstration of somebody else's work, or a presentation that restates the problem without a root cause or a proof. There is no partial credit and no conditional tier.

CONFIDENCE Established practice: the binary rule mirrors the programme's placement-eligibility rule

SkyWays in one picture

SkyWays Logistics moves full truckloads from factories in North and West India to receiving facilities. Your database is a frozen snapshot of two of those facilities at about 10:30 on 4 August 2026.



280 to 360
loads coordinated a day, company-wide

190 to 240
receiving appointments a day

15 to 25
driver exception messages on a normal day

20 to 35
messages inside 30 minutes during a disruption

5
operations coordinators per shift

21
live shipments in your snapshot

LEGEND Standard dock · Refrigerated dock · Heavy dock · Direction of freight

CONFIDENCE Established practice: the company-wide figures are the classroom operating profile, and the snapshot counts are from the database

Follow one truck

Six stages, six tables, and a pain point at every joint. A delay anywhere after stage three breaks a chain of commitments that several people made in good faith.

1 Shipment planned

shipments

shipment_id, origin, destination, product class, priority, original_eta_ts

2 Driver and vehicle assigned

drivers, vehicles

driver_id, vehicle_id, carrier_id, vehicle type and capacity

3 Appointment booked

appointment_slots, appointments

slot_id, dock, slot_start_ts, slot_end_ts, CONFIRMED

4 Truck travels

eta_updates, chat_threads

declared_eta_ts, confidence_code, delay_reason_code

5 Gate and yard

facility_checkins

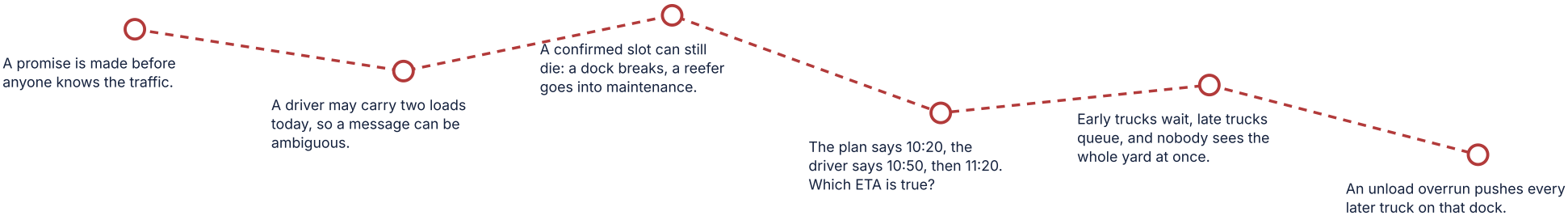
gate_in_ts, arrival_state, queue_state, queue_position

6 Dock and unload

facility_checkins, dock_status_events

dock_in_ts, unload_start_ts, unload_end_ts, gate_out_ts

WHERE IT GOES WRONG



THE MOVEMENT IN ONE LINE

Origin warehouse, then the truck in transit, then the destination gate, then the yard or waiting area, then the dock door, then unloading, then exit.
Every arrow in this chain is a commitment somebody else is relying on.

LEGEND Stage number — Where the plan goes wrong

CONFIDENCE Established practice: the stage model follows the freight-operations primer that accompanies the dataset

Who hurts, and how

Five parties see five different slices of the same truck. The exception is painful because no one of them holds the whole picture.

PARTY	WHAT IT KNOWS	WHAT IT NEEDS DURING AN EXCEPTION	WHERE IT HURTS TODAY
Driver	Physical location, breakdown, traffic, likely delay.	Clear choices, instructions and a confirmation.	Waits in the yard for hours with no answer; the detention is unpaid.
Dispatcher / carrier	Vehicle assignment and driver availability.	A revised plan and escalation status.	Learns about the missed slot after the customer does.
Operations coordinator	Shipment plan, customer commitment, open exceptions.	One consistent view across many simultaneous requests.	Handles 20 to 35 messages in 30 minutes by phone and memory.
Warehouse planner	Dock capacity, labour, equipment and local rules.	Feasible arrivals without overbooking.	Two trucks promised one dock; a reefer turned away at 18:30.
Consignee / customer	Inventory or production dependency.	Reliable revised arrival information.	A production line waits for a part that is in a yard 2 km away.

THE SHARED PAIN

Nobody in this table is careless. Each party sees one slice, and the slices are reconciled by a coordinator on the phone. Your Five Whys will almost always end at that reconciliation, not at a person.

LEGEND The pain as experienced today

CONFIDENCE Established practice: the five parties and what they know come from the operating brief; the hurt column is illustrative

Where the truth lives

Five systems hold five fragments of the answer. The job of any build in this capstone is to assemble the fragments before a single word goes back to the driver.



THE FRAGMENTS, ASSEMBLED

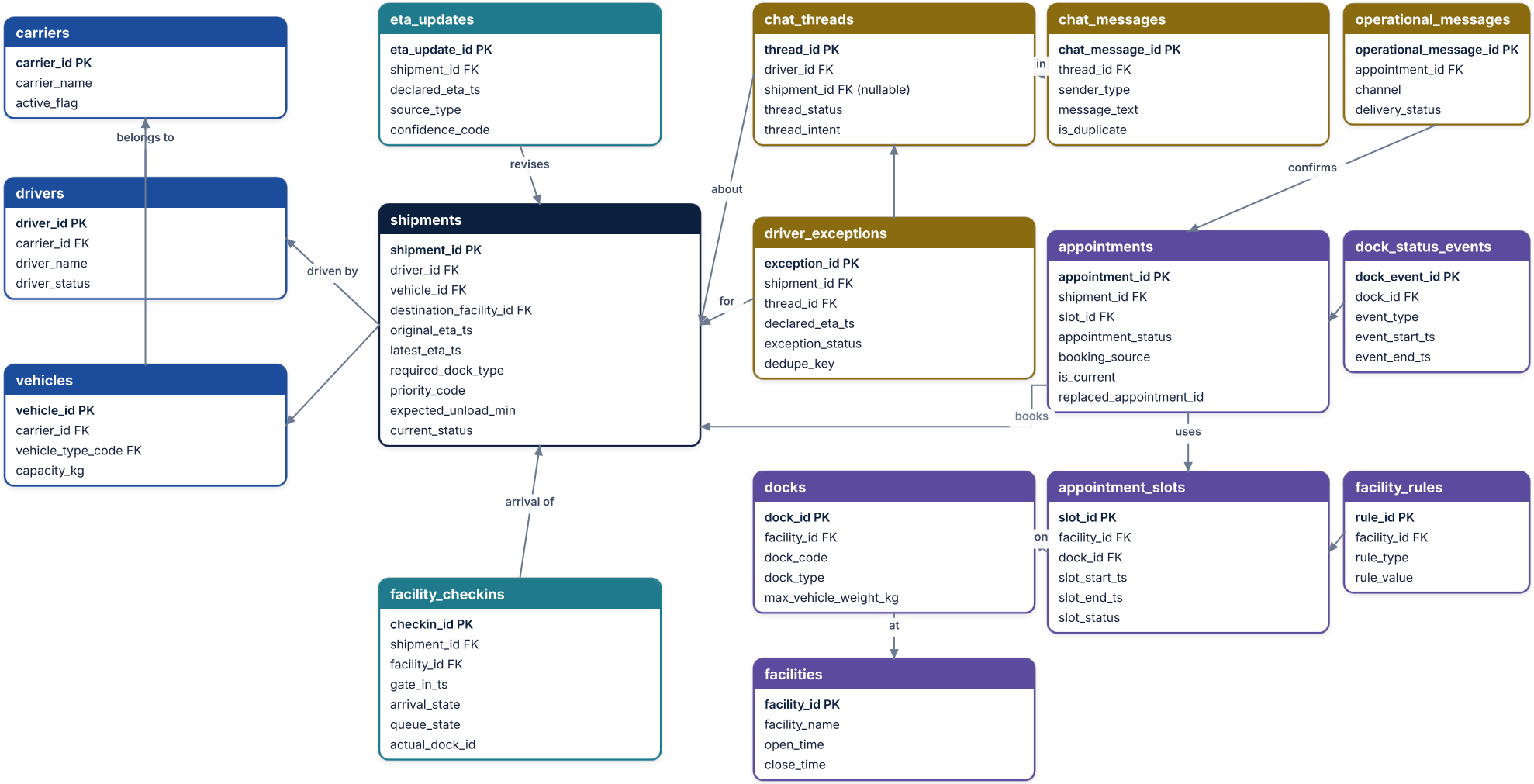
TMS: the plan said 10:20.
WMS: the slot is 10:00 to 11:00 on D4, CONFIRMED.
Messaging: nothing has been sent to the warehouse.
Chat: the driver now says 11:20, HIGH confidence.
Gate: no check-in row yet, so the truck has not arrived.

LEGEND – – – A fragment of the answer flowing in Table names in gold are the classroom tables for that system

CONFIDENCE Established practice: the five-system split is the operating brief's; the question in the centre is the SHP1006 case from the data

The data model

Sixteen of the eighteen tables, grouped by the system that owns them. The shipment is the hub: every other row either plans it, moves it, talks about it, or books it a door.



LEGEND Identity · Shipment hub · Movement · Conversation · Capacity and booking · Foreign key, arrow to parent

CONFIDENCE Established practice: names are those in data/skyways_data_dictionary.csv; vehicle_types and facility_contacts omitted for space

One message, traced

One short message touches seven tables before a trustworthy reply exists. If your design reads fewer than these, it is guessing somewhere.

MSG001 · THR001 · DRV006 Manoj Sharma · 09:34

"Traffic after Shahpura. Reaching around 11:20. Any slot after 12?"

TABLE	WHAT THE ROWS SAY RIGHT NOW	WHY THE BUILD NEEDS IT
chat_threads	THR001 links DRV006 to SHP1006 and is OPEN with intent REPORT_DELAY.	*Which load the message is about, so the question is not ambiguous.*
driver_exceptions	EXC001: TRAFFIC, reported delay 60 min, declared ETA 11:20, earliest acceptable 12:00, status NEEDS_INFORMATION.	*What was understood so far, and that a constraint is still missing.*
eta_updates	ETA005 at 08:50 said 10:50 (MEDIUM). ETA006 at 09:35 says 11:20 (HIGH). v_latest_eta picks 11:20.	*Which ETA is true now, and how much to trust it.*
shipments	SHP1006: medical devices from Ghaziabad, 7,800 kg on a 20 ft truck, HIGH priority, STANDARD dock, 45 minutes to unload, IN_TRANSIT.	*What any new slot must fit: dock type, duration, priority.*
appointments + appointment_slots	APT1006 is CONFIRMED on SLOT-JAI-045, D4, 10:00 to 11:00. The truck arrives at 11:20, so the slot is dead.	*The commitment that is about to break, and that it is still occupying capacity.*
facility_checkins	No row for SHP1006. The truck has not reached the gate.	*Arrival is a fact only when the gate records it, never when the driver says it.*
v_slot_availability	After 12:00 on standard docks: D2 12:00 AVAILABLE, D4 12:00 AVAILABLE, D1 12:00 OCCUPIED, D3 12:00 BLOCKED (leveller breakdown until 13:00), all four docks AVAILABLE at 13:00.	*The feasible choices the driver can actually be shown.*

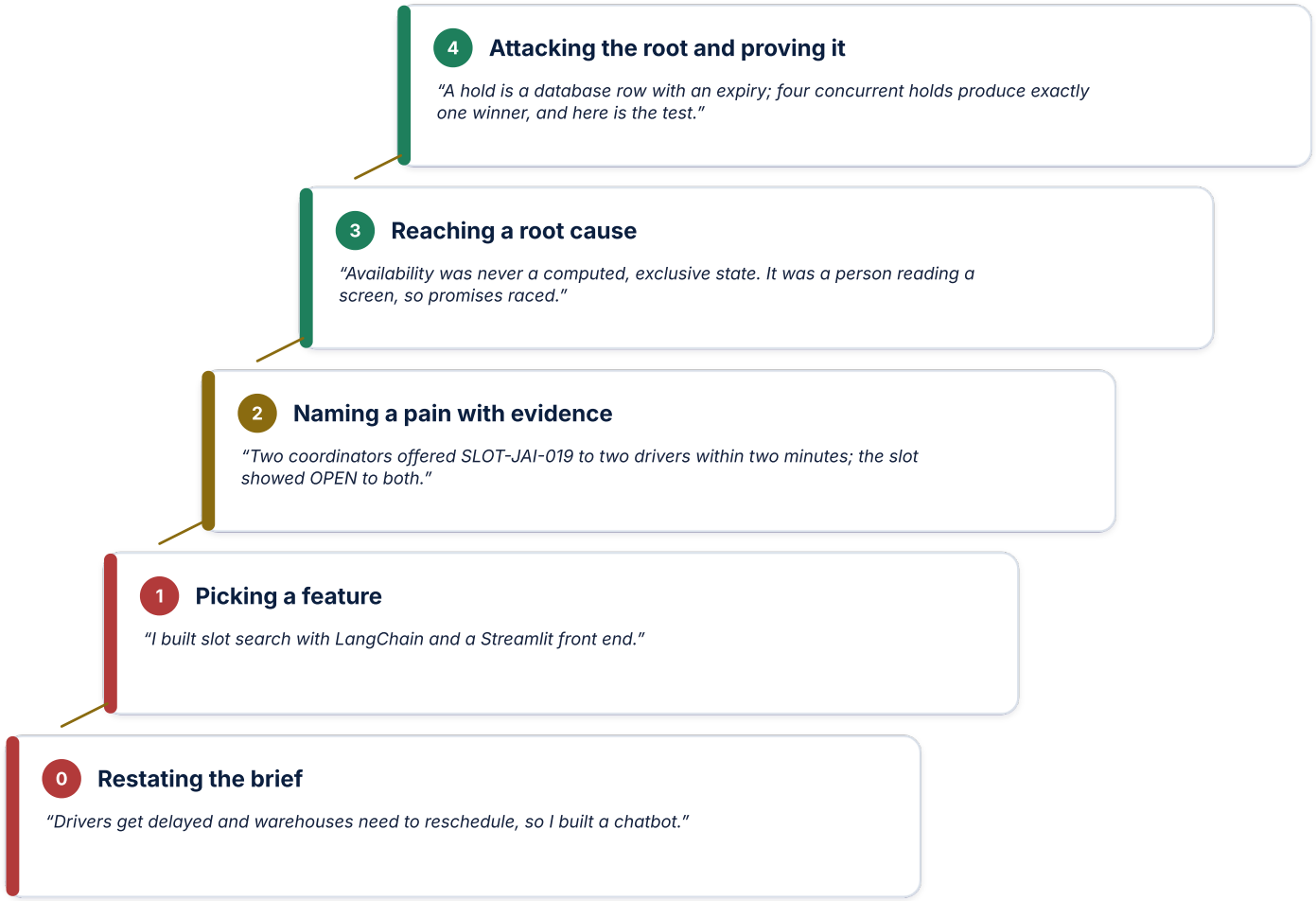
Trace this yourself in 03_explore/explore_skyways.ipynb before day two ends. The seven tables above are the minimum any build in this brief must read before it replies.

LEGEND Conversation · Movement · Shipment · Capacity and booking

CONFIDENCE Established practice: every value on this page is read from the snapshot database

Why the same problem again fails

Most capstones that fail stop at level 1: a feature built on a problem that was restated, never understood. Levels 2 to 4 are where this brief spends your first week.



WHAT THE PANEL HEARS

Level 4
The panel asks what you would do next. You have cleared.

Level 3
The panel starts asking design questions, which is where marks live.

Level 2
The panel leans in. You looked at the data.

Level 1
The panel hears a tool list. Any tutorial produces this.

Level 0
The panel hears the brief read back and learns nothing about you.

LEGEND Levels that do not clear · The turning point: evidence · Levels that clear

CONFIDENCE This brief's own construction: the five-level ladder is this brief's construction, built from the capstone panel's scoring habits

Five Whys, worked

Two chains from the same snapshot, one per track. Read each downward as "because" and upward as "therefore"; if a step does not survive both readings, it is not a why.

Agentic chain: the double promise

THE PAIN AS SEEN

Two drivers were offered SLOT-JAI-019 (D2, 12:00) within two minutes of each other.

1. **WHY**
   Two coordinators read the same screen, and it showed D2 12:00 as OPEN to both.
   Evidence: appointment_slots.slot_status; five coordinators per shift
2. **WHY**
   Showing a slot changes nothing in any system; only a confirmed booking does.
   Evidence: no appointments row exists until booking; OM001 SENT, OM002 QUEUED
3. **WHY**
   The warehouse confirmation travels by email with no expiry, so a slot stays half-promised.
   Evidence: APT1013A pending since 09:32, APT1014A since 09:52, OM004 FAILED
4. **WHY**
   The process was designed for 15 to 25 exceptions a day, spread out, with no idea of concurrency.
   Evidence: 20 to 35 messages inside 30 minutes during a disruption
5. **WHY**
   Availability was never a computed, exclusive state. It was a person's reading of a screen.
   Evidence: v_slot_availability had to be written for the classroom because no system computes it

ROOT CAUSE

Availability is not computed and not exclusive, so promises race.

ML chain: the overrun cascade

THE PAIN AS SEEN

An unload on D2 overran its slot and three later trucks on D2 queued in the yard (DEVT003).

1. **WHY**
   Every slot on D2 is 60 minutes, whatever the load.
   Evidence: facilities.default_unload_min = 60; appointment_slots are hourly
2. **WHY**
   The booked window uses the facility default, not the shipment's expected_unload_min.
   Evidence: shipments.expected_unload_min ranges from 45 to 90
3. **WHY**
   expected_unload_min is itself a guess typed at planning time, not an estimate from history.
   Evidence: dock_visit_history: unload_min from 20 to 392 minutes, median 43
4. **WHY**
   Nobody has measured what drives unload time at SkyWays.
   Evidence: pallet_count, dock_type, evening hour and labour shortage all move it
5. **WHY**
   Duration is assumed once and never predicted, so every slot is sized blind.
   Evidence: the baseline notebook: dock-type median already beats the fixed default

ROOT CAUSE

Unload duration is assumed, not predicted, so slots are sized blind.

LEGEND Agentic track chain · ML track chain · Root cause

CONFIDENCE Established practice: Five Whys is Toyota's root-cause method (Taiichi Ohno); these two chains are worked examples, and you will write your own

What to do at each why

Each level of the chain has its own question, its own research move and its own prompt. The decision column is where scope is born.

WHY LEVEL	THE QUESTION TO ASK	THE RESEARCH MOVE	THE PROMPT TO USE	WHAT YOU KEEP	DECISION
1 Symptom	What exactly happened, to whom, when? Which rows prove it?	Data probe: run one query that returns the offending rows.	**Finding what the data can and cannot answer** 01_discover/discovery_prompts.md	The query and its row count	Measure
2 Mechanism	Which process step produced that outcome? Who or what acted?	Driver voice: read the chat threads around the moment.	**Pushing a why deeper** 01_discover/discovery_prompts.md	The two or three candidate mechanisms, ranked	Mitigate
3 Design	What design choice made that step behave this way?	Vendor scan: how does a shipped product do this step?	**Checking a claim before you repeat it** 01_discover/discovery_prompts.md	A features table: we build, we fake, out of scope	Solve
4 Assumption	What did the designers assume about the world that no longer holds?	Paper skim: what does the literature say the problem really is?	**Understanding the domain** 01_discover/discovery_prompts.md	One sentence on what the paper decides and ignores	Measure or scope out
5 Root	What is the missing concept, state or measurement?	Interrogate your own design: argue against your root cause.	**Interrogating your own design** 01_discover/discovery_prompts.md	The root cause in one sentence, and its test	Solve

THE FOUR DECISIONS, AND THE RULE

Solve means your build changes it. Mitigate means a rule or a message reduces it. Measure means your build reports it without changing it. Scope out means you name it as a limit. A capstone solves one or two whys, mitigates one, measures one, and scopes the rest out in writing. Solving all five is the shape of a project that never ships.

LEGEND Solve in the build · Mitigate with a rule · Measure and report · Scope out, in writing

CONFIDENCE This brief's own construction: the level names and the four decisions are this brief's construction on top of the Five Whys

Pain-point depth matrix

Five candidates from the same snapshot, scored on the five criteria in the kit. The matrix exists to stop you choosing the pain point you noticed first.

CANDIDATE PAIN POINT	Frequency	Cost when it happens	Evidence in the data	Solvable in two weeks	Depth reached	TOTAL	VERDICT
A. Two drivers promised one slot	5	5	5	5	5	25	Agentic
B. Unload overruns cascade down a dock	3	5	5	5	5	23	ML
C. Ambiguous driver messages (two loads, one message)	5	3	3	3	3	17	Agentic, as a sub-feature
D. Reefer refused when D5 is down	1	5	5	3	3	17	Escalation case, not a capstone
E. Detention charges disputed with carriers	3	5	1	1	1	11	Future work

SCORING GUIDE (1, 3, 5)

Frequency	Cost	Evidence	Solvable	Depth
1 monthly	1 minutes of one person's time	1 none in the tables	1 needs data you do not have	1 one why
3 weekly	3 an hour of coordination or a missed slot	3 present in one table	3 needs one new table or tool	3 three whys
5 several times a day	5 detention, a broken promise, a lost dock hour	5 traceable across three or more tables	5 buildable with the kit as it is	5 five whys with evidence

DECISION RULE

Take the highest total whose Solvable score is 3 or 5. A candidate that scores high everywhere except Solvable becomes a future-work slide, never a capstone. Two candidates within two points of each other: pick the one whose root cause you can state in one sentence without notes.

LEGEND Points to the Agentic track · Points to the ML track · Not a capstone on its own

CONFIDENCE This brief's own construction: the five criteria and the 1-3-5 guide are this brief's construction; the scores shown are illustrative and yours will differ

From root cause to candidates

Three candidates per root cause, two checks per candidate. Only a candidate that attacks the root and fits two weeks becomes a PRD-lite; the rest become limits or future work.

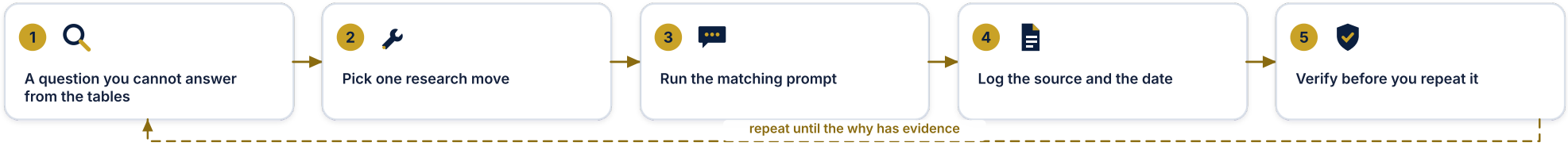


LEGEND Passes both checks · Check passed · Check failed

CONFIDENCE This brief's own construction: the two-check gate is this brief's construction; the candidates are worked examples for the two chains on page 13

Research moves and prompts

Research is a loop with a log, not a browsing session. Every answer you keep must name its source, its date, and the check you ran before trusting it.



THE FOUR RESEARCH MOVES

Vendor scan

Watch one dock-scheduling demo (Opendock, C3) and list every feature you see.

Keep: A table with three columns: we build, we fake, out of scope.

Paper skim

Read the abstract and the figures of one scheduling paper from Part H.

Keep: One sentence on what the paper decides and one on what it ignores.

Data probe

Run three queries that would confirm or refute your current why.

Keep: The queries and their row counts, pasted into the worksheet.

Driver voice

Read all 148 labelled messages once, slowly.

Keep: The five messages your current design would get wrong.

THE FIVE PROMPTS IN 01_DISCOVER/DISCOVERY_PROMPTS.MD, AND THE TRAP EACH ONE AVOIDS

Understanding the domain

Asks the assistant to answer as a warehouse operations manager and to cite the industry term for each stage.

TRAP AVOIDED
Vague domain words in your deck.

Pushing a why deeper

Three candidate root causes, ranked, each with the datum that would confirm or refute it, and no solutions yet.

TRAP AVOIDED
Jumping to a solution at why two.

Checking a claim

Argue against a belief you hold, with a two-driver, one-dock scenario.

TRAP AVOIDED
Repeating something that sounds right.

What the data can answer

Paste the data dictionary, get the SQL, and a list of what the data cannot tell you.

TRAP AVOIDED
Promising an analysis the tables cannot support.

Interrogating your design

The five questions a sceptical planner would ask, ranked by how embarrassing an unanswered one is.

TRAP AVOIDED
Meeting those questions for the first time in the demo.

LEGEND --- The loop runs again for the next why The trap each prompt exists to avoid

CONFIDENCE This brief's own construction: the four moves and five prompts are this brief's construction and ship in the kit

PRD-lite in one page

Eight slots, one page. If the filled version spills onto a second page, the scope is already too big for two weeks.

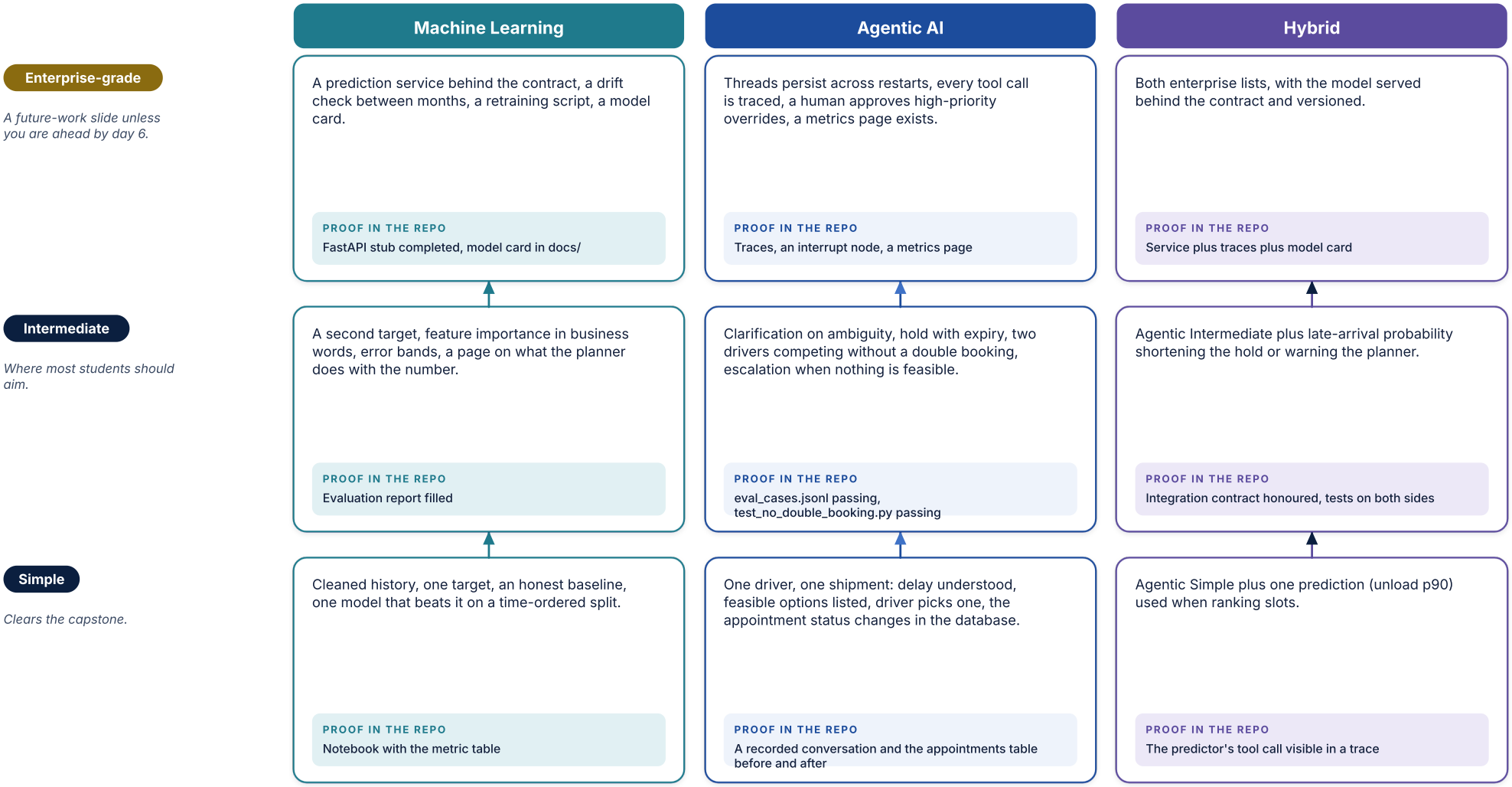
THE SKELETON (02_scope/prd_lite.md)	FILLED FOR CANDIDATE A, THE DOUBLE PROMISE
1 Problem *One sentence from the root cause, with the number that proves it matters.*	Two coordinators can offer the same dock slot to two drivers within two minutes because availability is a screen reading, not a computed state. On 4 August, SLOT-JAI-019 was offered twice.
2 Who it is for *Three users: what each does today and what changes for them.*	Driver: types a message, gets numbered options and a plain status. Coordinator: stops phoning for midday exceptions. Planner: sees holds, not surprises.
3 The one job *The single outcome the system produces. One sentence.*	Turn a free-text delay message into a held, feasible dock slot, requested from the warehouse, without ever double-booking a slot.
4 In scope *Three numbered items, each a thing the demo will show.*	1. Understand delay and constraint from text. 2. List feasible slots from v_slot_availability. 3. Hold with expiry, request confirmation, report status.
5 Out of scope and why *Two items, each with the reason it is out.*	1. Facility-wide rescheduling, because it needs a solver and a week on its own. 2. Priority overrides, because they are a planner's decision.
6 Success measures *Three rows: baseline today, target for the demo, how you measure it.*	Double bookings under 4 concurrent requests: today unknown, target 0, measured by replay_scenario.py. Clarification turns: target at most 1 per ambiguous message, measured on eval_cases.jsonl.
7 Guardrails *What the system never decides on its own.*	Never says CONFIRMED without the tool result. Never invents a slot. Never decides priority, safety or money.
8 Scope level *Simple, Intermediate or Enterprise, and one justifying sentence.*	Intermediate, because the hold, the concurrency test and the escalation path are all buildable with the kit by day 8.

LEGEND Slot number · A filled example, yours will differ

CONFIDENCE Established practice: a one-page PRD is standard product practice; the eight slots are this brief's selection

Scope ladder: simple to enterprise

Three rungs per track. Everything on the rung you pick is required, everything above it is a slide, and the proof column is what the panel opens first.



LEGEND Machine Learning · Agentic AI · Hybrid · Future work for almost everyone

CONFIDENCE This brief's own construction: the three rungs and their contents are this brief's construction, sized to the 35-hour budget

Guardrails

Three rings of decision. The language model lives in the smallest one, and every mark for architecture depends on you keeping it there.



THE AGENT NEVER DECIDES

- Whether two drivers can be promised the same capacity.
- Whether a vehicle is physically compatible with a dock.
- How scarce capacity is prioritised when business rules conflict.
- Whether a booking is committed in the system of record.
- Anything about driver safety, legal exposure, penalties or commercial terms.

THE AGENT ALWAYS DOES

- States the commitment status in the first line of every reply.
- Asks one question at a time, and only when the missing item blocks the next step.
- Hands over to a person when nothing is feasible, after two unanswered clarifications, or when the message is out of scope.
- Treats a request as a request until the tool result says CONFIRMED.
- Logs every tool call so the trace can be replayed.

FOR THE ML TRACK

The same rings apply with one substitution: the model estimates, the evaluation decides whether the estimate is trusted, and the planner decides what to do with it. A prediction is never a booking.

LEGEND What the model decides · What tools and the database decide · What people decide

CONFIDENCE Established practice: the never-decides list is taken from the operating brief for this dataset; the ring picture is this brief's rendering of it

Context diagram

Four kinds of people, four existing systems, and every arrow names what crosses the boundary.



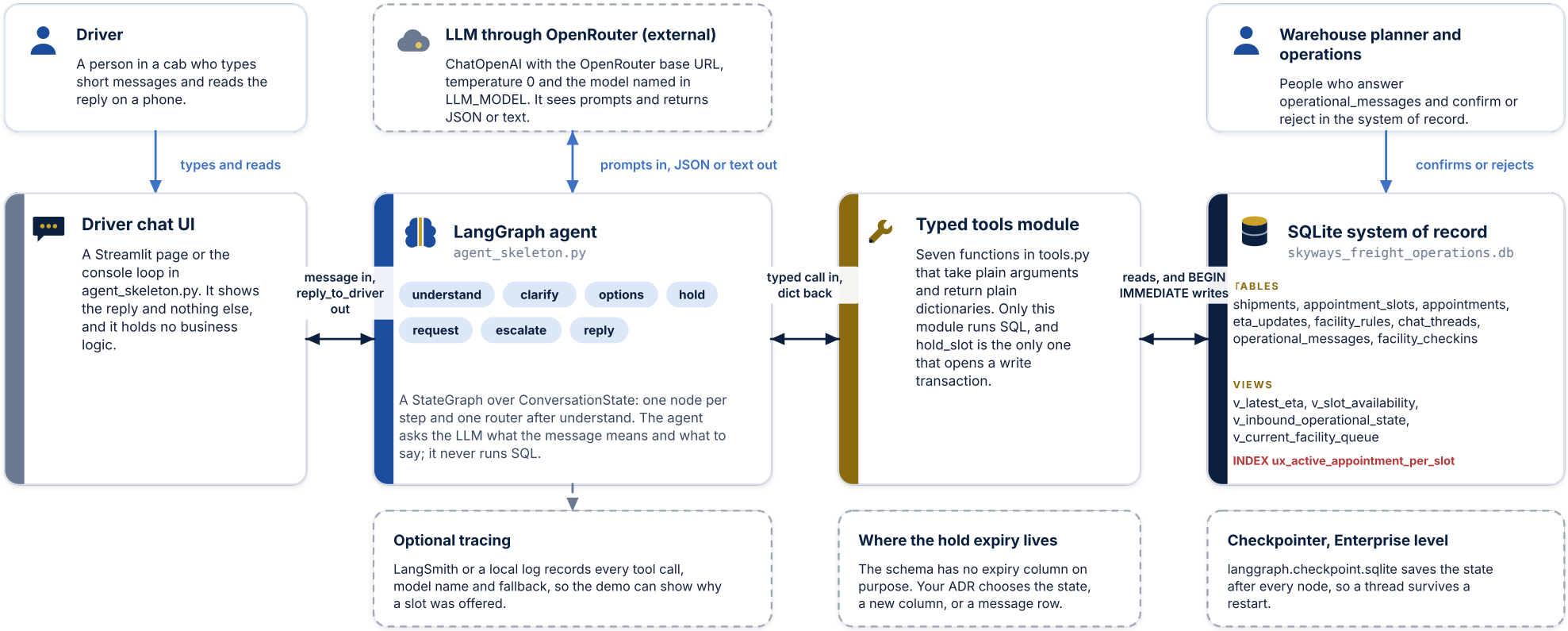
LEGEND the system you build · a person who talks to it · an existing system, tables in the kit · the flow your capstone changes · a flow you only read

CONFIDENCE Established practice: the context level is the first level of Simon Brown's C4 model; the boundary follows the tables in the kit

AGENTIC

Containers: Agentic track

Four containers and one external model. Every arrow carries typed data, so any box can be swapped alone.



WHO DECIDES WHAT

The LLM decides what to say and which tool to call.

It reads the driver's words, picks the intent, chooses the next tool and writes the reply. It is never asked whether a slot is free.

The tools decide what is true.

find_feasible_slots applies the six checks against the views and returns only what the data supports. A slot the tool did not return does not exist for the agent.

The database decides what is committed.

The unique index accepts one active appointment per slot and rejects the second. Until a row sits in appointments, nothing has been promised.

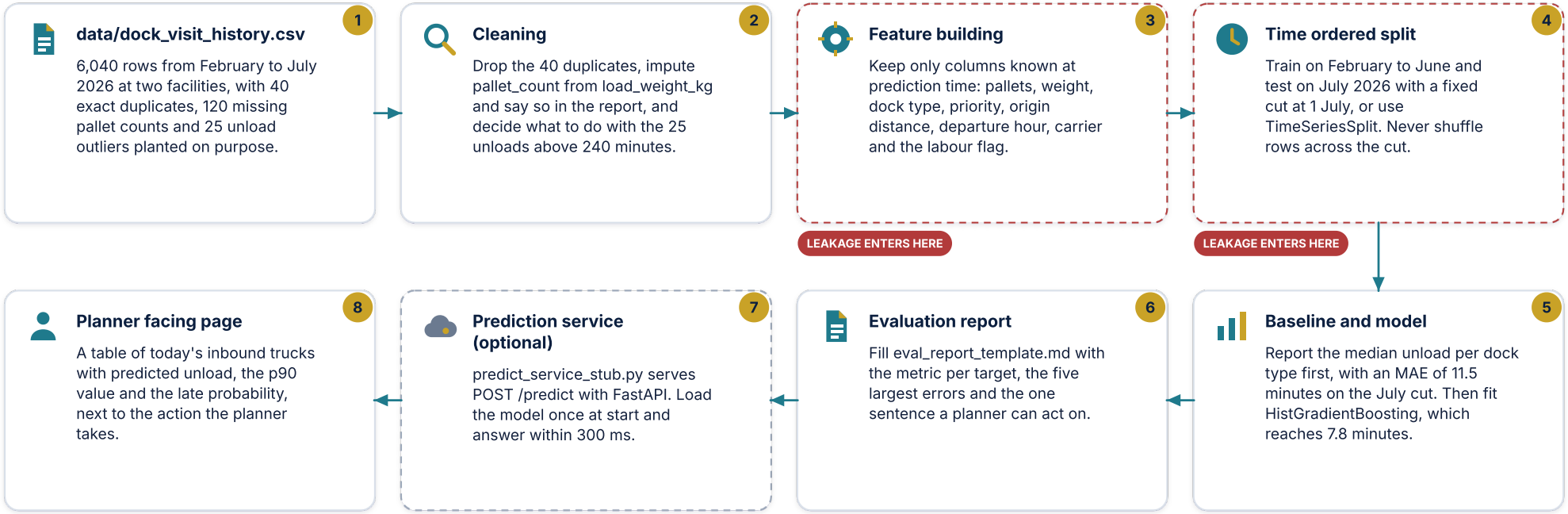
LEGEND the language model's decisions · the tools' decisions · the database's decisions · external or optional container

CONFIDENCE Established practice: the container level is the second level of the C4 model; the three way split of decisions is the rule tools.py states

ML

Containers: ML track

Eight steps in a snake. In the two outlined in red a model can quietly learn the answer itself.



WHERE LEAKAGE ENTERS, AND WHAT STAYS HONEST

A shuffled split sees the future

Shuffling puts July visits into training and tests on June ones, so the model has already seen the period it is judged on. The panel asks for the time cut first, every time.

Two columns are the answer in disguise

yard_wait_min and actual_gate_in_ts are written after the moment of prediction. Any feature built from them tells a lateness model when the truck arrived, which is the thing it is meant to predict.

The baseline keeps the model honest

The dock type median is reported first and the model must beat it on July rows it never saw. The gap from 11.5 to 7.8 minutes is the number the planner can feel.

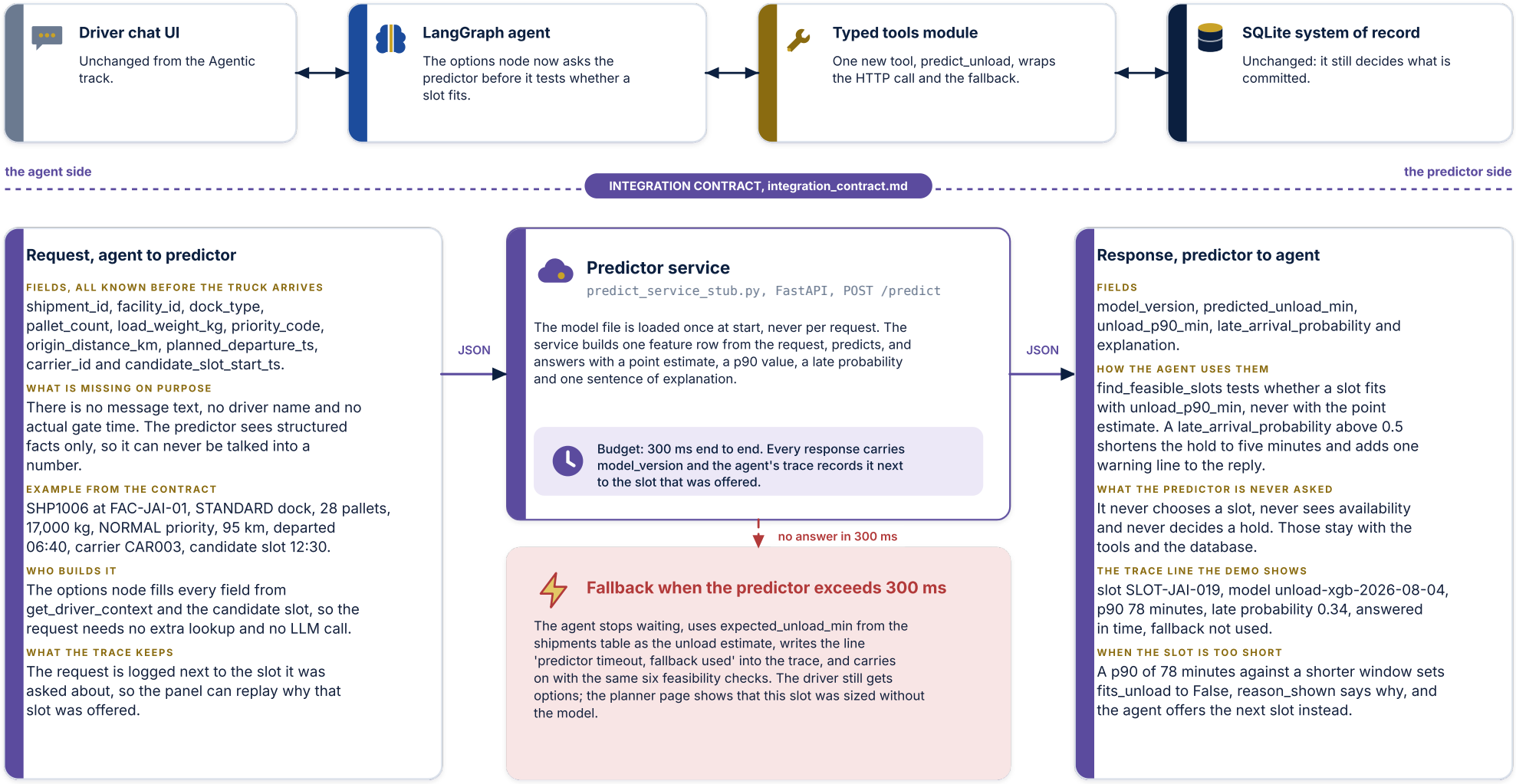
LEGEND a step you build · an optional step · a step where leakage can enter · a trap · the safeguard

CONFIDENCE Established practice: a baseline first and a time ordered split are established practice; the eight boxes mirror the kit notebook and templates

HYBRID

Containers: Hybrid track

The Agentic containers stay. The predictor joins behind one contract, and the agent survives its absence.



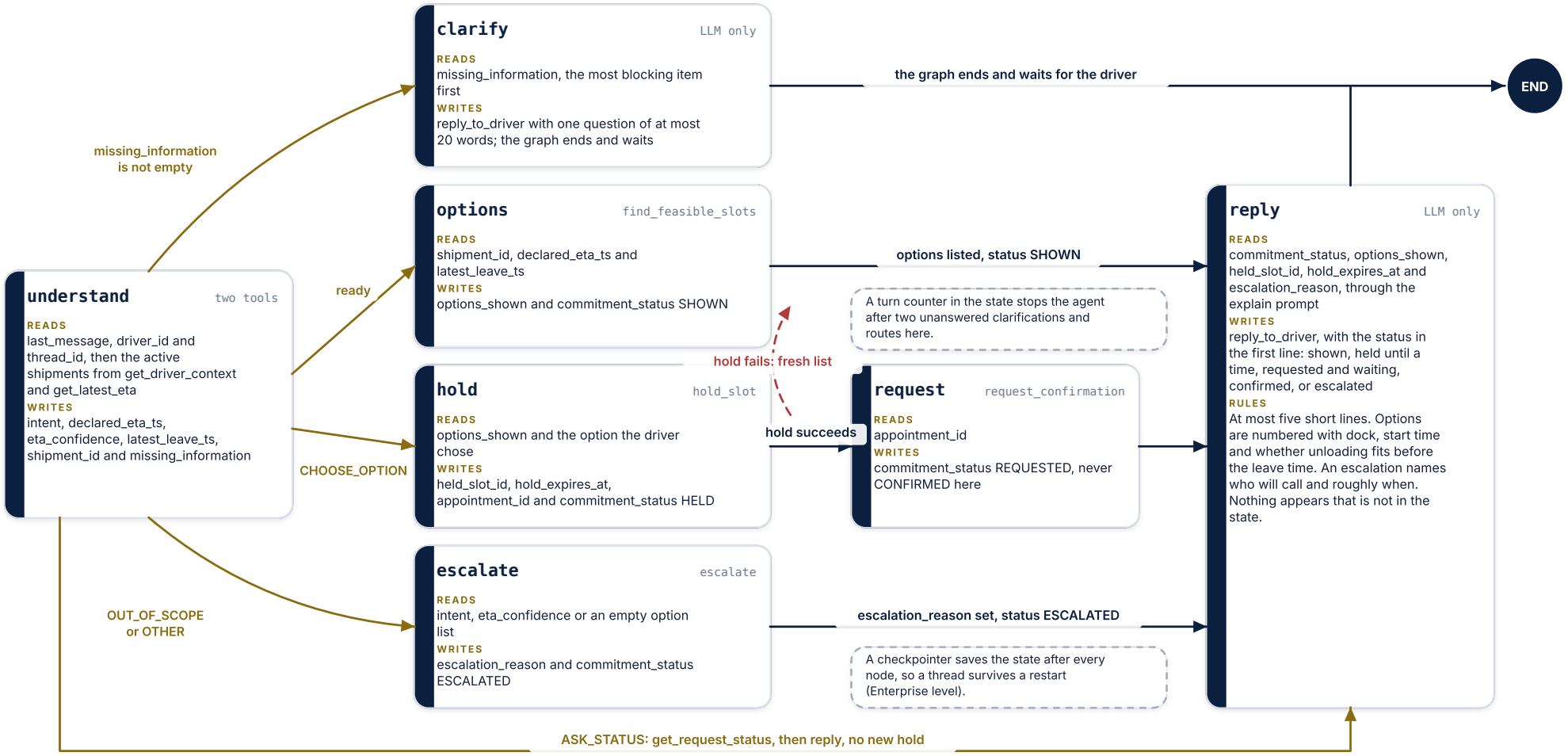
LEGEND the predictor and its contract · the contract boundary · the fallback path · the agent's decisions · the database's decisions

CONFIDENCE This brief's own construction: the JSON shapes, the 300 ms budget and the p90 rule come from the kit's integration contract; the fallback wording is this brief's

AGENTIC HYBRID

Inside the agent

Seven nodes, one router, one state object. Each node touches a few fields, so you can test it alone.



route_after_understand checks in this order: OUT_OF_SCOPE or OTHER first, then missing information, then CHOOSE_OPTION, then ASK_STATUS, and otherwise options. The order matters because an out of scope message must never reach a tool.

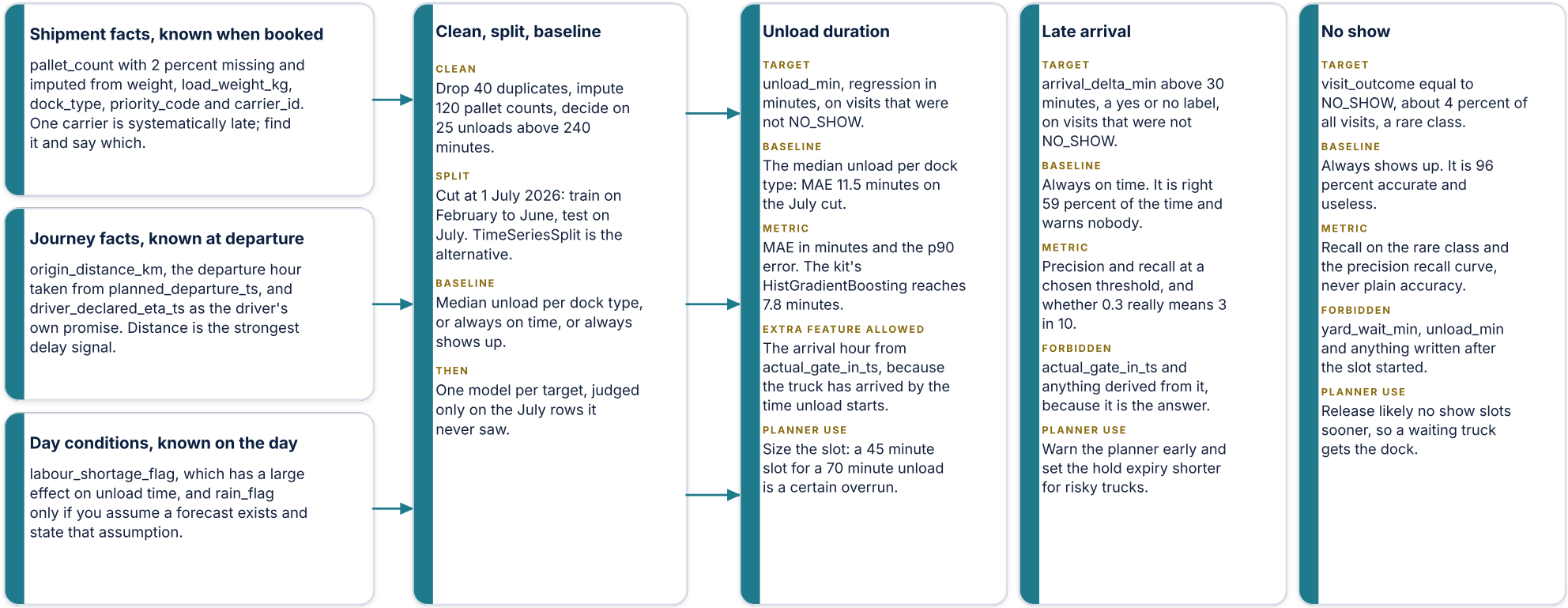
LEGEND a node in agent_skeleton.py, with the tool it calls — a routing edge with its condition — a fixed edge — the fallback edge — a TODO from the kit

CONFIDENCE Established practice: a StateGraph with conditional edges is the LangGraph pattern the kit ships; the field names come from state.py

ML HYBRID

Inside the ML pipeline

Three targets, one honest feature set, one baseline each. The left column is all a request may carry.



Forbidden at prediction time, for every target

yard_wait_min happens after arrival, so it leaks into both arrival and unload targets. actual_gate_in_ts leaks into the arrival targets and is allowed only for unload duration. detention_min and arrival_delta_min are outcomes, never inputs. If a feature would not exist at the minute the agent asks for a prediction, it does not belong in the request, and the panel will ask you to point at the column list that proves it.

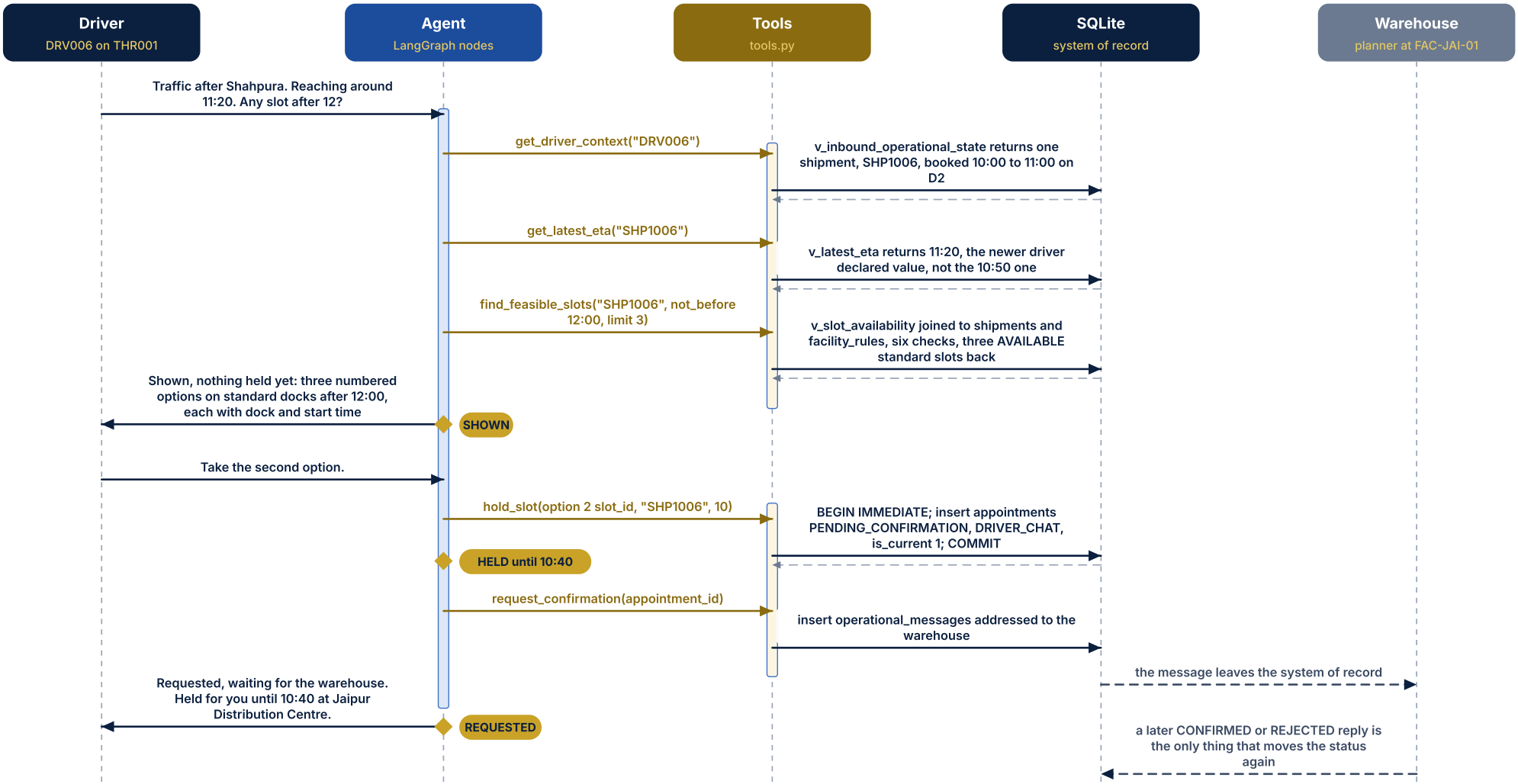
LEGEND a feature group or a target · columns that leak the answer · the flow of rows

CONFIDENCE This brief's own construction: the targets, leakage columns and baseline rule come from feature_spec.md; the three feature groups are this brief's

AGENTIC HYBRID

Sequence: delay to held slot

One message, three reads, one write, one request. The status changes three times; the driver hears each.



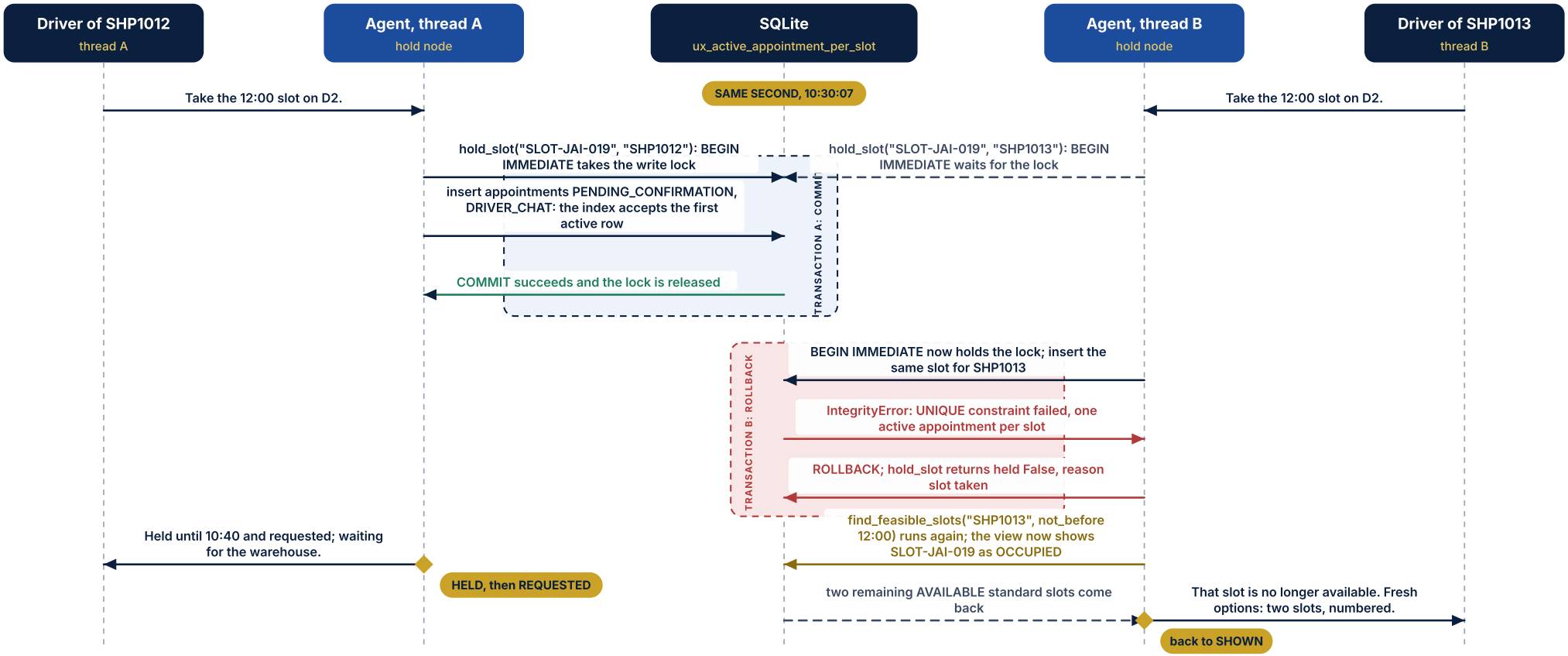
LEGEND a message or a query · a typed tool call · a result or a later reply · the commitment status changes here · the container is active

CONFIDENCE Established practice: the sequence diagram is UML notation; the messages follow cases EC01 and EC02; 10:40 assumes a ten minute hold placed at 10:30

AGENTIC HYBRID

Sequence: two drivers, one slot

Two holds arrive in the same second. The database picks the winner and the loser gets a fresh list.



WHAT THE TEST PROVES, test_no_double_booking.py and case EC11

Two threads attempt the same insert on a copy of the database. Exactly one wins and the check below returns 1. The loser is the database doing its job; the agent's only duty is to catch the IntegrityError and call find_feasible_slots again. At Intermediate level you add the test that the fresh list no longer contains the taken slot. At Enterprise level you add the test that an expired hold is released and the slot shows AVAILABLE again.

```
select count(*) from appointments where slot_id = 'SLOT-JAI-019' and is_current = 1 and appointment_status in ('PENDING_CONFIRMATION', 'CONFIRMED', 'IN_PROGRESS') must equal 1
```

LEGEND transaction A · transaction B · a commit · a rejection · a second tool call · the status changes here

CONFIDENCE Established practice: a unique partial index as the arbiter of concurrent writes is established; the scenario mirrors test_no_double_booking.py and EC11

The commitment state machine

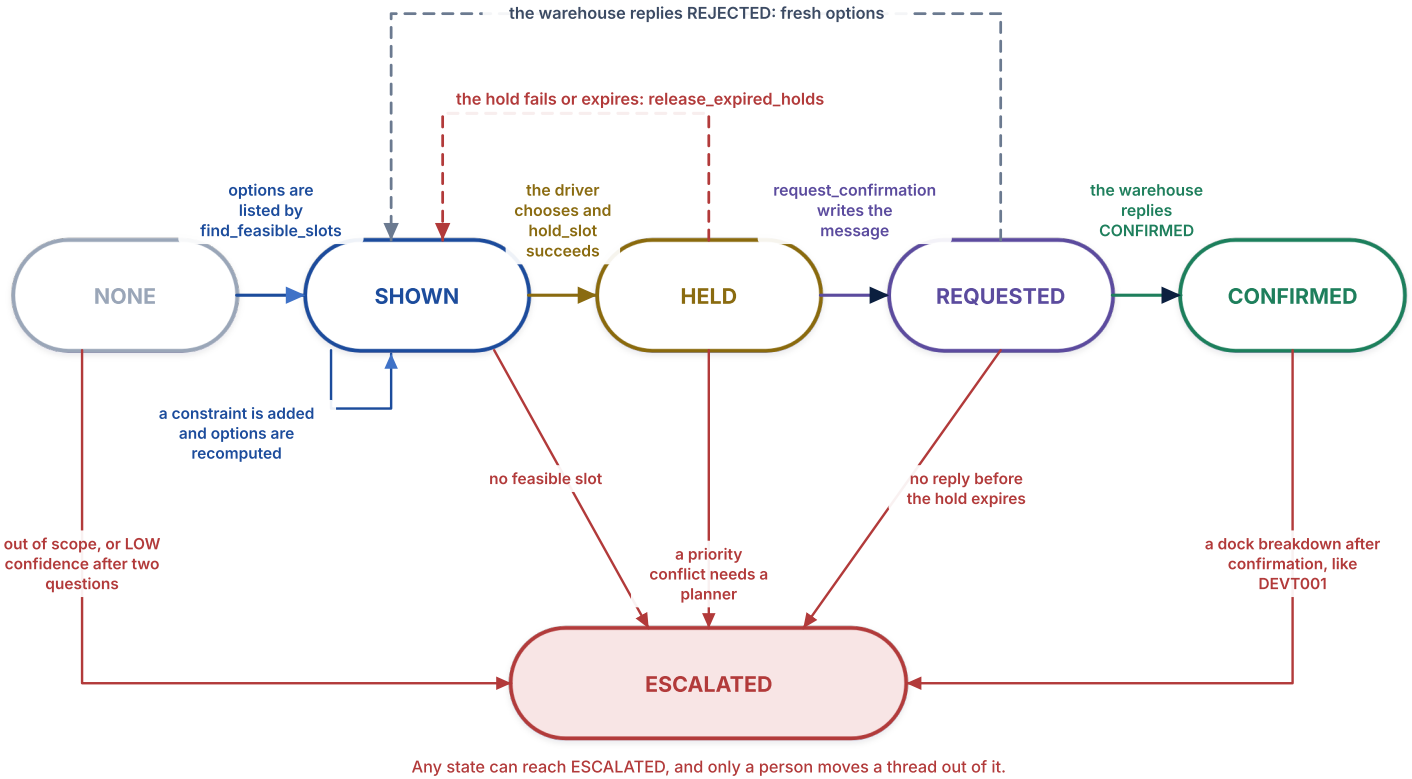
Six states. The driver hears a different sentence in each one, and that sentence is the product.

THE RULE

Showing is not holding, holding is not confirming, and a request is not a confirmation.

Why HELD can expire

The schema has no expiry column on purpose. Your ADR decides whether hold_expires_at lives in the state, a new column or a message row.



WHAT THE DRIVER IS TOLD IN EACH STATE

NONE. Nothing has been promised yet. The agent is still asking a question or looking up options.

SHOWN. Here are up to three options with dock and start time. None of them is yours until you choose one.

HELD. Option two is held for you until the stated time, and it will be released if the warehouse has not answered by then.

REQUESTED. The warehouse has been asked to confirm. The slot is still only held, so please wait for the next message.

CONFIRMED. The warehouse confirmed the slot. Check in up to 60 minutes early.

ESCALATED. This needs a person. Operations will contact you, and the agent says roughly when.

WHERE THE STATE LIVES

commitment_status in ConversationState is the agent's memory of it. The appointments row is the truth. When the two disagree, the row wins and the agent reads it back with get_request_status.

LEGEND forward, in the colour of the state entered · hold fails or expires · rejection · escalation

CONFIDENCE This brief's own construction: the state names are this brief's, bound in state.py; a request not being a confirmation is established

AGENTIC HYBRID

Tool contracts

Eight functions, each with a promise and a refusal. The LLM calls them by name and never sees the SQL.

get_driver_context(driver_id)

RETURNS
driver_id, active_shipments as a list of rows from v_inbound_operational_state, and needs_disambiguation.

GUARANTEES
Only shipments in ASSIGNED, IN_TRANSIT, AT_GATE or WAITING are returned, ordered by slot start. More than one row sets the flag.

REFUSES
It never guesses which shipment the driver means. DRV004 with SHP1004 and SHP1020 must be asked.

get_latest_eta(shipment_id)

RETURNS
The v_latest_eta row: the effective ETA, its source and its confidence.

GUARANTEES
The newest ETA wins, so SHP1006 gives 11:20 and never the earlier 10:50.

REFUSES
An unknown shipment returns an error field. It never invents a time.

find_feasible_slots(shipment_id, not_before_ts, latest_leave_ts, limit)

RETURNS
Up to limit SlotOption dicts: slot_id, dock_code, slot_start_ts, slot_end_ts, fits_unload and reason_shown.

GUARANTEES
All six checks hold: start after not_before, dock type matches, AVAILABLE in v_slot_availability, unload fits, before the 21:00 rule, done before latest_leave.

REFUSES
It never filters on slot_status alone, because a slot can be OPEN and still carry an active appointment.

hold_slot(slot_id, shipment_id, hold_minutes)

RETURNS
held True with appointment_id and the expiry, or held False with reason slot taken.

GUARANTEES
BEGIN IMMEDIATE, then one insert with PENDING_CONFIRMATION, DRIVER_CHAT and is_current 1. The IntegrityError from the index is caught, never raised.

REFUSES
It never returns CONFIRMED and never overwrites another active appointment.

request_confirmation(appointment_id)

RETURNS
appointment_id, status PENDING_CONFIRMATION and the message_id of the row written.

GUARANTEES
One operational_messages row addressed to the warehouse is written for every call.

REFUSES
It never moves the status to CONFIRMED. A failed send like OM004 is still not a confirmation.

get_request_status(appointment_id)

RETURNS
The current appointment_status and the latest warehouse reply from operational_messages, if any.

GUARANTEES
It only reads, so the answer always matches the system of record at that moment.

REFUSES
It never creates or extends a hold. Case EC07 checks that hold_slot is not called on a status question.

escalate(thread_id, reason)

RETURNS
What the driver should be told: who will contact them and roughly when.

GUARANTEES
chat_threads.thread_status is updated and one operational_messages row goes to operations. Used for no feasible slot, LOW confidence after two questions, OUT_OF_SCOPE and priority conflicts.

REFUSES
It never resolves the conflict itself and never leaks a contact's personal details.

release_expired_holds(now_ts)

RETURNS
How many PENDING_CONFIRMATION appointments were cancelled.

GUARANTEES
Every hold past its expiry is set to CANCELLED, so v_slot_availability shows the slot AVAILABLE again.

REFUSES
It never touches CONFIRMED or IN_PROGRESS rows. Where the expiry lives is your ADR, because the schema has no expiry column on purpose.

LEGEND finished for you in the kit · left for you with guidance · needs your ADR on where hold expiry lives

CONFIDENCE Established practice: typed tool contracts with explicit refusals are established practice; the eight names and signatures come from tools.py

Data and feature contracts

The left half says what the agent may trust as committed. The right half says what the model may know.

DATA CONTRACT THE AGENT RELIES ON

`v_slot_availability.availability_status`

AVAILABLE · OCCUPIED · BLOCKED · CLOSED

Only AVAILABLE counts as free. OCCUPIED means an active appointment already sits on the slot, BLOCKED means a dock_status_events row closed the dock, and CLOSED means the slot is outside facility hours or past the 21:00 rule.

`ux_active_appointment_per_slot`

One active appointment per slot, where active means appointment_status in PENDING_CONFIRMATION, CONFIRMED or IN_PROGRESS with is_current 1. The second insert raises IntegrityError inside the transaction, and hold_slot turns that into held False. The index is the only arbiter of a race, so your agent never needs a lock of its own.

`appointment_status` three active values in gold, four terminal values in grey

PENDING_CONFIRMATION · CONFIRMED · IN_PROGRESS · COMPLETED · CANCELLED · NO_SHOW · REJECTED

`booking_source` the agent always writes DRIVER_CHAT

PLANNER · DRIVER_CHAT · WAREHOUSE · SCHEDULING_TOOL · MANUAL_OVERRIDE

Views and rules the tools read

VIEWS

v_latest_eta gives the newest ETA with its source. v_inbound_operational_state gives one row per active shipment with its booked slot. v_current_facility_queue shows who is waiting at the gate.

FACILITY_RULES

CHECKIN_EARLY_LIMIT_MIN is 60. The no show limit is 30 minutes after slot start, 20 at Gurugram. LAST_NEW_START_TIME is 21:00. Reefer loads use D5 and loads above 25,000 kg use D6.

FEATURE CONTRACT THE MODEL RELIES ON

Known at the minute of prediction

pallet_count, load_weight_kg, dock_type, priority_code, carrier_id, origin_distance_km, the departure hour from planned_departure_ts, labour_shortage_flag, and rain_flag only with a stated forecast assumption. Nothing else enters the request.

Written after it, so it leaks

yard_wait_min for every target. actual_gate_in_ts for the arrival targets, allowed as the arrival hour for unload duration only. detention_min, arrival_delta_min and unload_min are outcomes, never inputs.

Integration JSON, integration_contract.md

REQUEST, AGENT TO PREDICTOR

`shipment_id`	string, SHP1006
`facility_id`	string, FAC-JAI-01
`dock_type`	STANDARD, REEFER or HEAVY
`pallet_count`	integer or null
`load_weight_kg`	integer
`priority_code`	string, NORMAL
`origin_distance_km`	integer
`planned_departure_ts`	ISO time with offset
`carrier_id`	string, CAR003
`candidate_slot_start_ts`	ISO time with offset

RESPONSE, PREDICTOR TO AGENT

`model_version`
string, recorded in the trace

`predicted_unload_min`
integer, the point estimate

`unload_p90_min`
integer, used to test fit

`late_arrival_probability`
0 to 1, above 0.5 shortens the hold

`explanation`
one sentence

Both sides keep four rules. The agent tests fit with the p90 value. A late probability above 0.5 shortens the hold to five minutes. The predictor answers within 300 ms or the agent falls back and says so. The predictor is never asked to choose a slot.

LEGEND database contract · the index and the leakage columns · feature contract · the integration JSON · an active status

CONFIDENCE This brief's own construction: the pairing is this brief's; every value comes from the kit's schema, feature_spec.md and integration_contract.md

AGENTIC HYBRID

Prompt collection

Four prompts, one job and one trap each. A rule enters a prompt only when a case fails without it.

system_prompt.md

Sets who the assistant is on every turn

JOB

It fixes the identity, the four things the assistant never does, and the way it speaks: short lines, 24 hour times with the facility name, and the commitment status every time.

RULES INSIDE

Never say confirmed unless the tool result says CONFIRMED. Never invent a slot, time or dock. Never decide safety, penalties or payment. Never compute availability yourself.

THE TRAP IT PREVENTS

The model declaring a slot confirmed because the driver sounded sure. Keep it under 350 words and add two passing exchanges as examples.

extraction_prompt.md

Turns the message into nine JSON fields

JOB

It returns intent, delay_minutes, revised_eta_text, shipment_reference, latest_time_constraint, option_choice, eta_confidence, needs_clarification and clarification_reason, and nothing else.

RULES INSIDE

A repair duration is not an arrival time, so revised_eta_text stays null and needs_clarification is true. A range like 1 to 3 hours is LOW confidence. Two active shipments and none named means ask which shipment.

THE TRAP IT PREVENTS

Tyre repaired in 40 minutes being read as an ETA, and a range being booked as a firm time.

clarify_prompt.md

Asks exactly one question

JOB

Given the one unclear thing, it writes one question of at most 20 words and offers the two most likely readings when the ambiguity is between readings.

RULES INSIDE

One question, not a list. Name the readings: does one hour late mean 11:00, or could the delay grow? Is this the SHP1012 load or the SHP1013 load?

THE TRAP IT PREVENTS

A wall of questions the driver answers with ok, which is intent OTHER and gets nothing done.

explain_prompt.md

Writes the reply the driver reads

JOB

From the facts only: the status, the facility, the options, the held slot and expiry, the escalation reason and the driver's constraint. At most five short lines.

RULES INSIDE

The commitment status is the first line. Options are numbered with dock, start time and whether unloading fits before the leave time. An escalation names who calls and roughly when.

THE TRAP IT PREVENTS

A status buried in line four, so the driver reads an option list as a booking and drives to the gate.

HOW TO ITERATE A PROMPT AGAINST eval_cases.jsonl

1 Run all 12

A short runner you write over eval_cases.jsonl prints pass or fail per case, the tools called and the final status.

→ **2 Read one failure**

Find the first case that fails and name the single rule the model broke, in one sentence.

→ **3 Change one rule**

Edit one line in one prompt file. Two edits at once means you cannot tell which one worked.

→ **4 Run all 12 again**

A fix that breaks another case is not a fix. EC12, the duplicate, breaks most often.

→ **5 Record it**

Add a line to docs/prompt_changelog.md: case, rule, before, after, cases now passing.

Temperature stays at 0 throughout, so a case that passes twice passes the third time, and a prompt that needs luck is a prompt that still has a missing rule.

LEGEND system prompt · extraction prompt · clarification prompt · explanation prompt · the iteration loop

CONFIDENCE This brief's own construction: the four prompts and their rules are in the kit; the trap per prompt and the five step loop are this brief's construction

Evaluation design

Twelve grouped cases judge the agent. A baseline, a time split and five explained errors judge the model.

AGENT EVALUATION, eval_cases.jsonl

Happy path
EC01 EC02 EC07
The THR001 delay gives SHOWN, the choice gives REQUESTED through hold_slot and request_confirmation, and a status question calls get_request_status without a new hold.

Clarification
EC03 EC04
Late by one hour gets one question and no tool call. DRV004 with two shipments is asked which one before anything else.

Constraint
EC05 EC08
Leave before 1:30 removes every option that ends after 13:30. An early arrival applies CHECKIN_EARLY_LIMIT_MIN and never displaces a HIGH or CRITICAL appointment.

Escalation
EC06 EC10
The reefer with D5 under maintenance finds no feasible slot and escalates. Detention charges are OUT_OF_SCOPE and go straight to escalate.

Concurrency
EC11
Two drivers take the same slot; exactly one hold succeeds and the loser's reply says no longer available.

Duplicate and reversal
EC12 EC09
The same message twice calls no tool the second time and shows options once. Do not book that releases the hold and returns to NONE or SHOWN.

THE PASS RULE

A case passes only when every field under expect holds: the tools in tools_called were called in that order, commitment_status equals the expected value, no string in must_not appears in the reply, and the db_check query returns 1. Eleven of twelve is a failing suite with a known gap, and you say which case and why in the README. Run the suite on a copy of the database, as test_no_double_booking.py does, so a failing case never dirties the demo state.

MODEL EVALUATION, eval_report_template.md

Baseline first
The median unload per dock type gives MAE 11.5 minutes on the July rows. For late arrival the baseline is always on time; for no show it is always shows up. The model is only interesting by the gap it opens.

Time split
Train on February to June, test on July 2026 with the cut at 1 July. State the row counts on each side and that the 40 duplicates were removed before the split, not after.

Metric per target
Unload duration: MAE and p90 error in minutes, 7.8 against 11.5 in the kit. Late arrival: precision and recall at the threshold you chose, plus calibration. No show: recall on the rare class and the precision recall curve.

The five largest errors
A table with the visit, the actual, the predicted, the error in minutes and your best reason. A pattern there, such as one carrier or the heavy dock, is worth more than a tenth of a point of MAE.

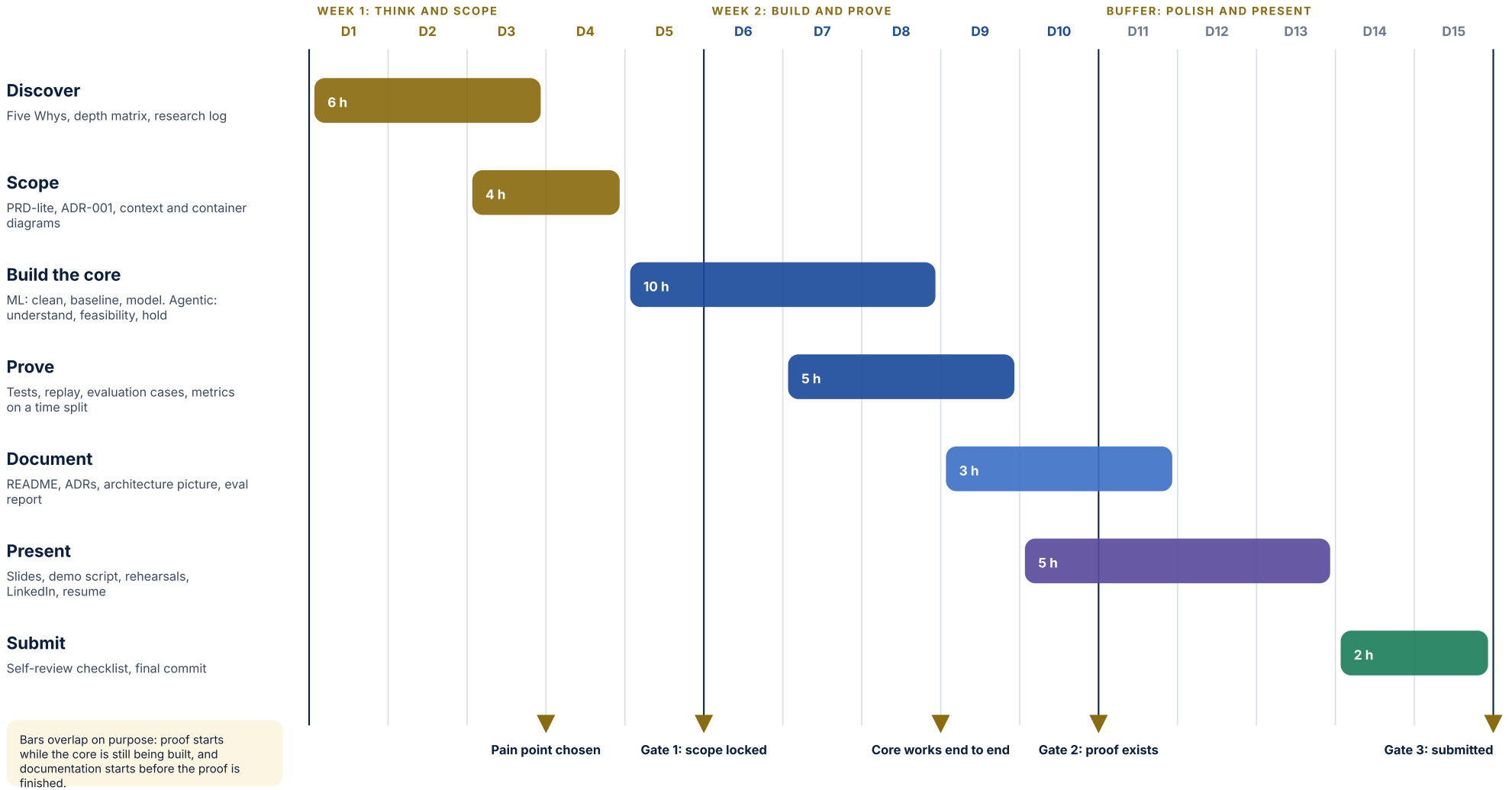
The planner's sentence
One line a planner can use without the notebook: on July visits the model never saw, the error is about 8 minutes, so a 15 minute buffer on the prediction covers most unloads.

LEGEND an evaluation case id · the pass rule · a model evaluation step

CONFIDENCE This brief's own construction: the six groups and the pass rule are this brief's reading of eval_cases.jsonl; baseline first and a time split are established

Two weeks on one page

Seven workstreams across fifteen days. The hours on each bar sum to the 35-hour budget; a bar that grows takes its hours from the one below it, never from the calendar.



LEGEND Thinking and scoping work · Building and proving · Documenting · Presenting · Submission

CONFIDENCE This brief's own construction: the stream split and hour allocation are this brief's construction

Day by day

Twelve rows, one tick box each, one commit message each. The definition of done is written before the day starts so that "almost" has nowhere to hide.

Day 1	☐ Kit runs, exploration notebook run end to end, three candidate pain points written down.	`git commit -m "day1: environment and first look at the data"`
Day 2	☐ Five Whys filled for all three candidates; at least two answers per sheet cite a query.	`git commit -m "day2: five whys"`
Day 3	☐ Depth matrix scored, one pain point chosen, PRD-lite drafted.	`git commit -m "day3: pain point chosen, prd-lite"`
Day 4	☐ Track and scope level fixed in ADR-001; context and container diagrams drawn; tool list or feature list fixed.	`git commit -m "day4: scope and architecture"`
Day 5	☐ Agentic: understand node and extraction prompt pass on 20 labelled messages. ML: cleaned data, baseline recorded.	`git commit -m "day5: first working piece"`
Day 6	☐ Agentic: find_feasible_slots is right for SHP1006 and for SHP1015 (none). ML: first model beats the baseline on a time split.	`git commit -m "day6: feasibility / first model"`
Day 7	☐ Agentic: hold and request work; the no-double-booking test passes. ML: second target or error bands; drivers explained.	`git commit -m "day7: commitment / second model"`
Day 8	☐ Agentic: EC03, EC04, EC06, EC10 pass. ML: evaluation report filled. Hybrid: the contract honoured end to end.	`git commit -m "day8: edge cases"`
Day 9	☐ Replay script demonstrates concurrency; all evaluation cases run; demo snapshot saved.	`git commit -m "day9: proof"`
Day 10	☐ README, architecture picture and ADRs in docs/; demo rehearsed once with a timer.	`git commit -m "day10: documentation and rehearsal"`
Day 11+	☐ Second rehearsal, LinkedIn post drafted, resume bullets written, one improvement if time remains.	`git commit -m "buffer: polish"`
Day 14+	☐ Self-review checklist passed, submission made.	`git commit -m "final"`

The rule when a day slips: if a row is not ticked by the end of the next day, cut scope one rung down the ladder. The calendar never stretches, and a smaller finished project beats a larger unfinished one every time.

LEGEND Think and scope days · Build and prove days · Buffer days

CONFIDENCE This brief's own construction: this is the file 06_check/definition_of_done_by_day.md, drawn

The starter kit

Eleven folders, three kinds of file. Open the kit's README first; it has the ten-step start and the rules the kit enforces.

FOLDER	WHAT IS INSIDE	WHEN	STATE
data/	SQLite database, schema and seed SQL, data dictionary, 6,040-row dock visit history, 148 labelled messages.	Day 1 onward	Finished for you
00_setup/	setup_check.py verifies Python, packages, the database, its views and the API key, and prints the fix beside every failure.	Day 1	Finished for you
01_discover/	Five Whys worksheet, pain-point depth matrix, discovery prompts, research log.	Days 1 to 3	Yours to fill
02_scope/	PRD-lite, ADR template, one filled ADR as an example, scope ladder checklist.	Days 3 to 4	Yours to fill
03_explore/	explore_skyways.ipynb traces THR001 through seven tables and ends at the competition and the no-answer case.	Day 2	Finished for you
04_build/agentic/	state.py, tools.py (two tools finished, five with guidance), agent_skeleton.py, four prompt files, eval_cases.jsonl, test_no_double_booking.py.	Days 5 to 9	Skeleton with TODOs
04_build/ml/	feature_spec.md, baseline_unload_duration.ipynb (baseline 11.5 min, model 7.8 min MAE), eval_report_template.md.	Days 5 to 9	Skeleton with TODOs
04_build/hybrid/	integration_contract.md with the request and response JSON, predict_service_stub.py.	Days 7 to 9	Yours to fill
04_build/scripts/	replay_scenario.py fires four holds at one slot and prints one winner; snapshot_db.py saves and restores the demo state.	Days 8 to 10	Finished for you
05_present/	Ten-minute outline, demo script, README template, LinkedIn post, resume bullets, panel questions.	Days 10 to 13	Yours to fill
06_check/	Definition of done by day, final self-review checklist.	Every day	Finished for you

Nothing in the kit is a finished answer. The finished pieces are the ones that stop a known mistake: the environment check, the exploration trace, the concurrency test, the replay, the baseline. Everything that carries your thinking is yours.

LEGEND Runs as shipped · A skeleton with TODO breadcrumbs · A template you fill

CONFIDENCE Established practice: every path and count on this page is the kit as shipped

Mistakes we expect

Twelve mistakes that past capstones made, each with the symptom you will see and the file in the kit that prevents it. Read this page again on day five and on day nine.

1 Shuffled train-test split

HOW YOU WILL KNOW
The model looks 40 percent better than it is because July rows leak into training.

THE FIX IN THE KIT
TimeSeriesSplit or a 1 July cut; baseline beside the model.

ML, Hybrid

2 The LLM computes availability

HOW YOU WILL KNOW
The reply lists a slot that the view says is OCCUPIED.

THE FIX IN THE KIT
The LLM chooses the tool; find_feasible_slots decides what is free.

Agentic, Hybrid

3 CONFIRMED without a tool result

HOW YOU WILL KNOW
The driver is told it is booked; the warehouse never saw a request.

THE FIX IN THE KIT
Status comes only from get_request_status; REQUESTED until then.

Agentic, Hybrid

4 A hold with no expiry

HOW YOU WILL KNOW
One abandoned conversation blocks D2 12:00 for the whole afternoon.

THE FIX IN THE KIT
Decide where expiry lives (ADR), run release_expired_holds.

Agentic, Hybrid

5 State only in Python memory

HOW YOU WILL KNOW
A restart forgets the hold; the database still has the row.

THE FIX IN THE KIT
The database is the system of record; add a checkpointer at Enterprise.

Agentic, Hybrid

6 Leakage through yard_wait_min

HOW YOU WILL KNOW
Perfect lateness prediction, because waiting happens after arriving.

THE FIX IN THE KIT
Use only columns known at prediction time (feature_spec.md).

ML, Hybrid

7 Duplicates and outliers ignored

HOW YOU WILL KNOW
40 duplicate rows inflate the test set; 25 outliers wreck the mean error.

THE FIX IN THE KIT
Drop duplicates, decide on the outliers, write the decision down.

ML, Hybrid

8 Demo against the live database

HOW YOU WILL KNOW
The second rehearsal finds the slot already held by the first.

THE FIX IN THE KIT
snapshot_db.py save before, restore after every run.

All

9 Coding on day one

HOW YOU WILL KNOW
A chatbot by day 3, a root cause by never.

THE FIX IN THE KIT
Days 1 to 4 produce documents, not code; Gate 1 enforces it.

All

10 Scope creep into Hybrid

HOW YOU WILL KNOW
Two half projects and no proof for either.

THE FIX IN THE KIT
Hybrid only if Agentic Simple is done by day 6.

All

11 Secrets in the repository

HOW YOU WILL KNOW
A key in .env is committed and revoked within the hour.

THE FIX IN THE KIT
.env in .gitignore; setup_check.py reads it, never prints it.

All

12 Numbers without a file

HOW YOU WILL KNOW
A slide says 92 percent; nobody can find where it came from.

THE FIX IN THE KIT
Every number on a slide points at a file in the repo.

All

LEGEND The symptom · The fix · Track labels in the corner of each card

CONFIDENCE Established practice: these mistakes were observed in earlier capstone panels; the fixes are the kit files

Ten minutes, eight slides

The timings are hard limits. No slide carries more than 30 words outside its picture, and every number on a slide exists in a file in your repository.



1 The pain

ON THE SLIDE

One scene: the rainy evening, four trucks, two docks, the coordinator on the phone, and one number.

WHAT YOU SAY

The problem as the planner feels it, with the number.

60 seconds

2 The root cause

ON THE SLIDE

Your Five Whys as a vertical chain with the root highlighted.

WHAT YOU SAY

Read the chain upward as "therefore".

60 seconds

3 What I built

ON THE SLIDE

The C4 container diagram of your system.

WHAT YOU SAY

Name each box and one sentence on what it owns.

60 seconds

4 The decision that mattered

ON THE SLIDE

Your most important ADR as a three-option table.

WHAT YOU SAY

Why you chose, and what it cost you.

60 seconds

5 Live demo

ON THE SLIDE

The running system, with a screenshot fallback behind it.

WHAT YOU SAY

Follow the demo script exactly.

180 seconds

6 Proof

ON THE SLIDE

The test output or the metric table with its baseline.

WHAT YOU SAY

One claim, one number, one file name.

60 seconds

7 What it cannot do yet

ON THE SLIDE

Three limits, honestly stated.

WHAT YOU SAY

Where the enterprise version differs.

30 seconds

8 What I would do next

ON THE SLIDE

Two or three improvements with effort in days.

WHAT YOU SAY

Close on the one you would start tomorrow.

30 seconds

LEGEND Problem and cause · Design · Demonstration · Proof · Limits and next steps

CONFIDENCE This brief's own construction: the eight-slide spine is this brief's construction, matched to the panel's scoring columns

The demo script

Six moves in three minutes, each with the words you say and the thing the panel must see. Rehearse it three times from a restored snapshot.

	0:00	0:40	1:10	1:40	2:20	2:50
YOU TYPE OR CLICK	Driver message: “Traffic after Shahpura. Reaching around 11:20. Any slot after 12?”	Driver message: “I need to leave by 1:30.”	Driver message: “Take the first one.”	Run replay_scenario.py in the terminal.	Driver message: “Evening traffic. ETA 6:30. Can the reefer unload tonight?”	Stop.
YOU SAY WHILE IT RUNS	*A real message from the dataset. Watch which tools fire.*	*A constraint. The list shrinks without me asking anything again.*	*Held, not confirmed. The warehouse still has to say yes.*	*Four trucks want the same slot at once. The database allows exactly one.*	*No feasible slot. The agent does not invent one; it escalates with a reason.*	*That is the loop: understand, option, hold, request, and refuse when it should.*
THE PANEL SEES	**Options listed, status SHOWN.**	**One option drops out.**	**appointments gains a PENDING_CONFIRMATION row.**	**One HELD, three REJECTED.**	**Status ESCALATED.**	

IF ANYTHING FAILS

Switch to the screenshots behind slide 5, say “here is what you would have seen”, and keep the timing. A demo that recovers in ten seconds scores higher than one that never fails.

ML TRACK VARIANT

Replace steps 1 to 3 with the evaluation table, one prediction explained with its drivers, and the planner action it changes. Keep step 4 if your model feeds slot sizing.

LEGEND Elapsed time at the start of the move · The recovery branch · The ML substitution

CONFIDENCE This brief's own construction: this is 05_present/demo_script.md, drawn; the messages are rows from the dataset

Your repository

A recruiter spends 40 seconds on a repository. The tree on the left and the README order on the right are built for those 40 seconds, and for the panel's 10 minutes.

`README.md`	The front door: problem, picture, demo, proof, limits. Written last, read first.
`docs/`	
`architecture.png`	The C4 container diagram exported from your tool of choice.
`five_whys.md`	Your filled worksheet, evidence and all.
`prd_lite.md`	The one page.
`adr/`	ADR-000 to ADR-00n. The panel reads these to see how you think.
`eval_report.md`	ML and Hybrid: the baseline, the split, the metric, the five worst errors.
`src/`	
`agent/ or model/`	Your code, with the kit skeleton filled in and your TODOs removed.
`tools.py`	Typed contracts; the LLM never touches SQL.
`prompts/`	The four prompt files, versioned.
`tests/`	
`test_no_double_booking.py`	The test the panel asks for.
`test_eval_cases.py`	Runs eval_cases.jsonl and prints a pass table.
`data/`	The snapshot database and the two CSVs. Never the demo copy with test rows.
`scripts/`	replay_scenario.py, snapshot_db.py, and anything you added.
`.env.example`	Variable names only. .env itself is in .gitignore.
`requirements.txt`	Loose pins, installable in one command.

README ANATOMY, TOP TO BOTTOM

1. **Title and one line**
   What it does and for whom. No adjectives.
2. **Architecture picture**
   Above the fold. A reader decides in five seconds whether to scroll.
3. **The problem**
   Two sentences with the number. Link to docs/five_whys.md.
4. **What it does**
   Three bullets in the user's words, not the stack's.
5. **Demo**
   A 60-second GIF, then the three commands that reproduce it.
6. **Proof**
   A two-column table: claim, evidence file or command output.
7. **Limits and next steps**
   Three honest limits, two next steps with effort.
8. **Data**
   Synthetic classroom dataset, snapshot 4 August 2026, schema link.

Commit at the end of every working day with the message from the day-by-day page.

LEGEND Folder File

CONFIDENCE Established practice: the README order is standard portfolio practice; the tree mirrors the kit

LinkedIn and resume

The post shows the scene and the proof, the bullet carries a verb and a number, and the spoken version is what you say when someone asks "what did you build?"

THE LINKEDIN POST, ANNOTATED

I just completed my capstone for the PG Diploma in AI-ML and Agentic AI Engineering.	*Say what it is in one line. The programme name does the credibility work.*
The problem: two coordinators could promise one dock slot to two drivers within two minutes, because availability was a screen reading.	*A sentence a logistics manager recognises, with the mechanism, not the tool.*
What I built: a tool-calling agent that turns a free-text delay message into a held, feasible dock slot without ever double-booking.	*The one job. No stack list here.*
What I learned the hard way: showing a slot and holding a slot are different states; a shuffled split made my model look 40 percent better than it was; Hybrid is two projects, so I did one well.	*Three lessons, each specific enough that a peer could reuse it.*
Proof: four concurrent holds on one slot produce exactly one winner, and the test is in the repo.	*One number or one test, never an adjective.*
Repository and architecture picture in the comments.	*The link goes in the first comment; the post carries the picture.*

THE RESUME BULLET FORMULA

Verb	What	For whom	Outcome	How
Built, designed, trained	a tool-calling agent	for truck drivers and dock planners	zero double bookings in 12 of 12 cases	LangGraph, OpenRouter, SQLite

- Built a tool-calling conversational agent (LangGraph, OpenRouter, SQLite) that converts free-text driver delay messages into feasible, held dock appointments; 12 of 12 evaluation conversations pass, including a concurrent-request case with zero double bookings.
- Trained a gradient-boosted regressor on 6,000 dock visits to predict unload duration, cutting median absolute error from 11.5 minutes (dock-type median baseline) to 7.8 on a time-ordered split, enabling 15-minute slot buffers instead of 30.

Numbers come from your repository only. Replace these with yours.

THE THIRTY-SECOND SPOKEN VERSION

The business problem was *one sentence with a number.*

I found the root cause was *one sentence, no tools named.*

I built *one sentence, the one job.*

The thing I am proudest of is *the proof.*

If I had another month I would *the one next step.*

Practise it until you can say it without notes, then shorten it by a third.

LEGEND Post text Italic notes explain why each line is there

CONFIDENCE Established practice: the verb, what, for whom, outcome, how formula is the common resume convention (the XYZ formula); the texts are templates in the kit

Facing the panel

Ten questions the panel asks almost every time. The weak answers are real ones from earlier panels; the good column tells you what to have ready.

	THE QUESTION	THE ANSWER THAT LOSES MARKS	WHAT A GOOD ANSWER CONTAINS
1	**Why this pain point?**	*"It seemed interesting."*	The Five Whys chain, the matrix score, the number.
2	**Walk me through the architecture.**	*Reads the boxes aloud.*	What each box owns and the one decision that shaped it.
3	**How does the agent know a slot is free?**	*"It checks the database."*	The six feasibility rules, the view used, the difference between OPEN and AVAILABLE.
4	**What stops a double booking?**	*"The code checks first."*	The unique index, the transaction, the test, the replay output.
5	**How did you evaluate the model?**	*"Accuracy was 92 percent."*	The baseline, the time-ordered split, the metric in minutes, where it fails.
6	**What did you not build?**	*Silence.*	Three named limits and the enterprise version of each.
7	**What was the hardest bug?**	*"Nothing major."*	A specific moment, what you saw, what you changed.
8	**What would you do with another month?**	*A feature list.*	One improvement, its effort, and why it comes first.
9	**What did the LLM get wrong?**	*"It was mostly fine."*	An evaluation case that failed first and the prompt or tool change that fixed it.
10	**How much did a conversation cost?**	*Unknown.*	Tokens per turn times price, and the monthly figure at 300 exceptions a day.

The questions that separate students most are 3, 4 and 5. Prepare them with the actual file open, and answer by showing rather than describing.

LEGEND Answers that lose marks · Contents of a strong answer

CONFIDENCE Established practice: the questions are the panel's standard probes; the weak answers are paraphrased from earlier sittings

How you are assessed

Seven columns totalling 100, each mapped to one of the three pillars. Track depth means real engineering on your chosen track, and it is scored separately from whether everything shipped.

15	20	20	15	10	10	10
Problem and solution framing	Architecture	Track depth	Demonstration	Rigour	Communication	Questions and answers

COLUMN	PILLAR IT TESTS	WHAT EARNS THE MARKS
Problem and solution framing · 15	Repository and defence	The root cause is evidenced and the solution attacks it. Framing is scored on its own, independent of whether the build shipped.
Architecture · 20	Repository	Boxes own decisions; the model sits in the smallest ring; ADRs show options considered.
Track depth · 20	Repository	Agentic: real tool design, state handling and concurrency. ML: honest evaluation and interpretation. Scored on whether real engineering happened, not on polish.
Demonstration · 15	Demonstration	The demo script runs, a refusal is shown, the fallback exists.
Rigour · 10	Repository	Tests, replay, time split, duplicates handled, numbers traceable to files.
Communication · 10	Demonstration	Eight slides, timed, one idea each, plain words.
Questions and answers · 10	Defence	The widest-spread column and the real separator between candidates.

HOW THE PANEL READS THE SCALE

The practical ceiling is about 80 of 100; nobody is awarded the top of a band. Deployment and test claims are checked against the repository during the sitting. A panel note is three to six sentences naming a specific moment and ending on the fix or the unanswered question.

HOW CLEARED IS DECIDED

Cleared or not cleared is read from the three pillars on page 5, with the rubric converting the sitting into the capstone mark recorded on your transcript. The programme office sets and communicates the threshold mark; this brief does not.

LEGEND Read from the repository · Observed in the demonstration · Tested in the defence · Both repository and defence

CONFIDENCE Established practice: the seven columns and weights are the programme's capstone rubric; the mapping to pillars is this brief's reading of it

Room to innovate

Ten directions past the Enterprise rung, ordered by effort from the bottom. Each is a fair answer to the panel question about what you would build next, and the one day rungs fit inside the buffer week.

	DIRECTION	WHAT IT DOES AND WHY IT MATTERS	EFFORT	EXTENDS
10	**Facility wide rolling horizon rescheduling with OR-Tools CP-SAT**	Model trucks as jobs and docks as machines, re-solve the next four hours whenever an ETA changes, and let the agent offer slots from the solver's plan.	6 days	Extends both
9	**No show prediction that releases slots early**	A classifier on visit_outcome flags the appointments likely to end as NO_SHOW, so the facility can release their slots before the 30 minute rule would.	4 days	Extends ML
8	**Predicted unload p90 feeding slot sizing**	A quantile model on unload_min returns the 90th percentile for this truck, and the agent sizes the slot window from it instead of from the fixed dock type default.	3 days	Extends both
7	**Hinglish robustness and voice input**	Add Hinglish variants of the 148 labelled messages, measure intent accuracy again, and accept a voice note through speech to text before the same classifier.	3 days	Extends Agentic
6	**Priority policy as explicit rules with a fairness audit**	Write the SHP1009 versus SHP1014 rule as data rather than prose, then audit six months of history for carriers who waited longer than their priority_code justified.	2 days	Extends both
5	**An MCP server that exposes the tools**	Wrap find_feasible_slots, hold_slot and the rest as MCP tools so any client can book through them. Opendock shipped exactly this for warehouse appointments.	2 days	Extends Agentic
4	**LangGraph interrupt for human approval of CRITICAL overrides**	The graph pauses before any MANUAL_OVERRIDE booking and resumes only when a planner approves it, which keeps the agent out of the priority call it must never make.	2 days	Extends Agentic
3	**Planner dashboard with detention minutes**	A Streamlit page over v_current_facility_queue and dock_visit_history shows detention_min by dock and hour, so the planner sees waiting time in time to act.	2 days	Extends ML
2	**Langfuse tracing and cost per conversation**	Every prompt, tool call and token count lands in one trace, so you can say what one driver conversation costs and where its latency goes.	1 day	Extends Agentic
1	**Hold expiry and release worker**	A background job moves every HELD slot back to SHOWN when its timer runs out, so a driver who went quiet cannot block capacity for the next truck.	1 day	Extends Agentic

Enterprise level, page 19. The scope ladder ends here and this one starts. Climb from rung 1, and stop when the buffer week runs out.

LEGEND Extends the ML track · Extends the Agentic track · Extends both tracks · Effort in working days for one person

CONFIDENCE This brief's own construction: the ordering and the day estimates are for one person, and a day means about two and a half focused hours

Datasets to augment

Nine datasets to set beside dock_visit_history.csv: one to warm up on, one to stress a method, none to pad a slide.

Delivery truck trips data — CC BY-SA 3.0

https://www.kaggle.com/datasets/ramakrishnanthiyagu/delivery-truck-trips-data

GPS pings from Indian full truckload trips, with Planned_ETA beside actual_eta and an on time or delay flag on every trip.

About 6,900 rows and 31 columns.

ADDS HERE It gives your ETA model real Indian road timing to test the driver_declared_eta_ts feature against.

Logistics Operations Database 2022 to 2024 — MIT

https://www.kaggle.com/datasets/yogape/logistics-operations-database

A synthetic Class 8 trucking company spread over 14 tables, including a Delivery Events table that records detention.

More than 85,000 records.

ADDS HERE A second schema with detention events lets you rehearse the detention_min analysis before you present it on SkyWays data.

DataCo Smart Supply Chain — CC0

https://www.kaggle.com/datasets/shashwatwork/dataco-smart-supply-chain-for-big-data-analysis

Order level records carrying a Late_delivery_risk label, which is the closest public cousin of your LATE outcome.

About 180,000 rows and 53 columns.

ADDS HERE A large labelled late delivery problem on which to practise the time split and the baseline comparison at scale.

E-Commerce Shipping Data — SEE PAGE

https://www.kaggle.com/datasets/prachi13/customer-analytics

A small shipment table with a Warehouse_block column and a Reached.on.Time target that is clean enough to model in an hour.

10,999 shipments and 12 columns. Check the licence on the page.

ADDS HERE A one hour classification baseline that warms you up before you open dock_visit_history.csv.

Logistics and supply chain dataset — CC0

https://www.kaggle.com/datasets/datasetengineer/logistics-and-supply-chain-dataset

Hourly synthetic operations rows that include an ETA Variation column and a Loading/Unloading Time column.

About 32,000 rows.

ADDS HERE An unloading time column from another source lets you compare its distribution with your unload_min before you trust either.

Smart Logistics Supply Chain Dataset — CC0

https://www.kaggle.com/datasets/ziya07/smart-logistics-supply-chain-dataset

A compact table with Waiting_Time and a Logistics_Delay_Reason column that names why a shipment slipped.

About 1,000 rows.

ADDS HERE The delay reason vocabulary is a ready made label set for the driver_exceptions table in your database.

Delivery Logistics Dataset India Multi-Partner — CC BY 4.0

https://www.kaggle.com/datasets/kundanbedmutha/delivery-logistics-dataset-india-multi-partner

Synthetic Indian deliveries across several partners, each with an expected time, an actual time and a Delay flag.

25,000 deliveries.

ADDS HERE The multi partner structure mirrors your four carriers, so you can test whether a carrier feature carries signal elsewhere.

Supply Chain Order Delay Risk Analysis — CC0

https://www.kaggle.com/datasets/jayjoshi37/supply-chain-order-delay-risk-analysis

Orders scored for delay risk, small enough to read end to end in one sitting.

2,800 orders.

ADDS HERE A set this small is ideal for a feature leakage hunt, where you find the column that quietly encodes the answer.

NYC Taxi Trip Duration — COMPETITION

https://www.kaggle.com/competitions/nyc-taxi-trip-duration

The canonical trip duration regression, with pickup and drop coordinates and the duration you must predict.

1.46 million rows. Competition data, so accept the rules on the page.

ADDS HERE The standard ETA regression warm up, where the time split and the log target you will reuse on unload_min come from.

Papers, docs and videos

Seven papers, nine docs and twelve videos, each with the reason it is here and its address in full.

PAPERS AND REPORTS

Dock Assignment and Truck Scheduling Problem, Monemi and Gelareh, 2023, arXiv. The formal model your feasibility tool approximates.
https://arxiv.org/abs/2311.02609

Integrated trucks assignment and scheduling with mixed service mode docks, Li and others, 2024, arXiv. Your D5 and D6 question in general form.
https://arxiv.org/abs/2412.09090

A comprehensive review of truck appointment scheduling models, Abdelmagid, Gheith and Eltawil, 2022, Transport Reviews. The map that places your project among systems already built.
https://doi.org/10.1080/01441647.2021.1955034

Data-driven multi-agent decision support for time slot management at container terminals, Nadi and others, 2023, arXiv. A deployed Rotterdam slot system, the closest cousin of the Hybrid track.
https://arxiv.org/abs/2311.15298

New research documents substantial financial and safety impacts from truck driver detention, ATRI, September 2024. Detention at 39.3 percent of stops and USD 11.5 billion lost, the numbers for your first slide.
https://truckingresearch.org/2024/09/new-research-documents-substantial-financial-and-safety-impacts-from-truck-driver-detention/

ReAct: Synergizing Reasoning and Acting in Language Models, Yao and others, 2022, arXiv. The thought, tool call and observation loop your agent skeleton runs.
https://arxiv.org/abs/2210.03629

Tool Learning with Large Language Models: A Survey, Qu and others, 2024, arXiv. How tool contracts and error handling are studied, so you can name what you did.
https://arxiv.org/abs/2405.17935

DOCS AND BLOGS

LangGraph overview, LangChain docs. Nodes, edges and state in one read before agent_skeleton.py.
https://docs.langchain.com/oss/python/langgraph/overview

Tools, LangChain docs. How a typed function becomes a tool the model can call.
https://docs.langchain.com/oss/python/langchain/tools

Interrupts, human in the loop, LangGraph docs. The pause and resume for approval of CRITICAL overrides.
https://docs.langchain.com/oss/python/langgraph/interrupts

Persistence, LangGraph docs. Checkpoints and threads, so a conversation survives a restart.
https://docs.langchain.com/oss/python/langgraph/persistence

Building effective AI agents, Anthropic. The case for simple patterns over frameworks, read before you add a node.
https://www.anthropic.com/research/building-effective-agents

Job shop scheduling, Google OR-Tools. Trucks as jobs and docks as machines, rung 10 in runnable Python.
https://developers.google.com/optimization/scheduling/job_shop

Dock scheduling, Opendock. What a shipped dock appointment product offers, so you know the bar.
https://opendock.com/products/dock-scheduling

Cross-validation and TimeSeriesSplit, scikit-learn. The split that keeps July out of training, with the leakage reasoning.
https://scikit-learn.org/stable/modules/cross_validation.html

Observability overview, Langfuse. Traces, spans and cost per run, the rung 2 setup in an afternoon.
https://langfuse.com/docs/observability/overview

VIDEOS: THE DOMAIN

The EASY way to understand loading docks!, Warehouse Buyers Club. Dock doors and bays in plain sight.
https://www.youtube.com/watch?v=EGpGKUVVsoQ

Receiving Operations Explained, Warehouse Inbound Process, Ops Evolution. Gate in to unload complete, step by step.
https://www.youtube.com/watch?v=WOQlw76IB14

What are TONU, Detention and Layover?, Never Stop Trucking. The driver's side of the fees your project avoids.
https://www.youtube.com/watch?v=izh0UL8y-_4

Truck Appointment System, Introduction and User Guide, APM Terminals. A live appointment system seen from the booking screen.
https://www.youtube.com/watch?v=BrvNS67l6Qo

VIDEOS: THE PROBLEM

Navigating Detention Times, FreightWaves. Why detention persists when everyone wants it gone.
https://www.youtube.com/watch?v=S4jWYJHANr4

Opendock: A Dock Scheduling Software for Warehouses, Loadsmart. The product your Enterprise rung is compared with.
https://www.youtube.com/watch?v=j9uTHo4nJKY

Opendock MCP Demo: AI Agent Creates Warehouse Appointments in Seconds, Loadsmart. An agent booking a dock through MCP, rung 5 done.
https://www.youtube.com/watch?v=aR5XzFZIK9A

The 5 Whys Explained, Root Cause Analysis, EPM. The method behind Part C in a few minutes.
https://www.youtube.com/watch?v=t7FcK8jV2yA

VIDEOS: THE STEPS

Building Effective Agents with LangGraph, LangChain. The patterns from the Anthropic essay, built live.
https://www.youtube.com/watch?v=aHCDrAbH_go

XGBoost Part 1: Regression, StatQuest. Boosted trees on a duration target, from the first tree.
https://www.youtube.com/watch?v=OtD8wVaFm6E

How to give a kick-ass demo, Ignite Sydney. The three minute live demo and what to do when it breaks.
https://www.youtube.com/watch?v=Cxl_3ANnE0A

How To Write a USEFUL README On Github, Learn Fast Make Things. The README that lets a stranger run your repository.
https://www.youtube.com/watch?v=E6NO0rgFub4

Tools and credits

Everything here is free or nearly free. Read the catch under each one before you depend on it during the demo.

Google Cloud free tier
https://cloud.google.com/free

YOU GET USD 300 of trial credit plus the always free products, more compute than this capstone needs.

THE CATCH The trial credit expires on a timer and a forgotten VM burns it quietly, so set a budget alert on day one.

Gemini API free tier
https://ai.google.dev/gemini-api/docs/pricing

YOU GET A free tier on the Gemini models with no card attached, enough for the Agentic track's evaluation runs.

THE CATCH Requests are rate limited per minute and per day, and free tier prompts may be used to improve the product, so send only synthetic data.

OpenRouter free models
https://openrouter.ai/models?q=free

YOU GET One key and one OpenAI compatible endpoint across many models whose free variants cost nothing to call.

THE CATCH Free variants come and go, and a model that answered yesterday can be gone today, so keep a second model configured.

OpenRouter limits
https://openrouter.ai/docs/api-reference/limits

YOU GET The numbers in one place: 20 requests a minute on free variants, 50 a day without credits, 1,000 a day with ten dollars of credit.

THE CATCH Fifty requests a day disappears inside one evaluation run, so read this page before you plan the run.

Hugging Face Spaces
https://huggingface.co/docs/hub/spaces-overview

YOU GET A free CPU Basic Space that hosts a Streamlit or Gradio app on a public address.

THE CATCH A Space goes to sleep after a period of inactivity and takes a while to wake, so open it before the panel sits down.

Streamlit Community Cloud
https://docs.streamlit.io/deploy/streamlit-community-cloud

YOU GET Free hosting for a public Streamlit app straight from your GitHub repository, with secrets kept out of the code.

THE CATCH Apps sleep when unused and resources are modest, so keep the model file small and load it once.

Render free tier
https://render.com/docs/free

YOU GET 750 free instance hours a month for a web service, enough to run a FastAPI tool server all month.

THE CATCH The service spins down after 15 minutes idle and the first request after that waits through a cold start.

Railway
https://railway.com/pricing

YOU GET A 30 day trial with five dollars of credit, enough to deploy the agent and its SQLite file for the demo.

THE CATCH The trial ends on its own and the credit is small, so use it for the demo week only.

GitHub Student Developer Pack
https://education.github.com/pack

YOU GET Credits and free tiers across many cloud and tooling partners for verified students.

THE CATCH Verification needs a student email or ID and can take days, so apply in week one.

anthropics/skills
https://github.com/anthropics/skills

YOU GET Example skill folders that show how to package instructions and scripts an agent can load.

THE CATCH It is a reference for structure, not a dock scheduling starter, so copy the shape and not the content.

langchain-ai/react-agent
https://github.com/langchain-ai/react-agent

YOU GET A minimal ReAct agent in LangGraph with tools, state and a configuration to swap for yours.

THE CATCH It ships with a web search tool you must remove, because your tools read SQLite and nothing else.

langchain-ai/new-langgraph-project
https://github.com/langchain-ai/new-langgraph-project

YOU GET The blank LangGraph layout with a graph, tests and a langgraph.json for the local studio.

THE CATCH It assumes the LangGraph CLI and Studio, which add setup time before a first node runs.

langchain-ai/langchain-academy
https://github.com/langchain-ai/langchain-academy

YOU GET The notebooks behind the free LangGraph course, from a first graph to persistence and interrupts.

THE CATCH Notebooks are not a repository structure, so port the ideas into modules before you submit.

awesome-LangGraph
https://github.com/vonzosten/awesome-LangGraph

YOU GET A curated list of LangGraph projects, templates and tutorials to borrow patterns from.

THE CATCH Lists age fast and some links point at older APIs, so check the LangGraph version before you copy.

BEFORE YOU SPEND MONEY

The free tiers on this page cover the whole capstone, including the demo. OpenRouter free models are limited to 50 requests a day without credits, so either buy ten dollars of credit once or use the Gemini free tier. Never put a key in a repository; keep it in a .env file that .gitignore excludes.

PG DIPLOMA IN AI-ML AND AGENTIC AI ENGINEERING · GUIDED CAPSTONE

Find the real pain, reach its cause, and build one thing that holds.

The whole brief in one sentence, and the whole capstone in one promise.

Nothing you present will be judged on how much it does. It will be judged on whether the problem was real, whether the cause was reached, whether the thing you built attacks that cause, and whether you can prove it in a file somebody else can run.

Indian Institute of Technology Gandhinagar

Programme delivered with Futurense Technologies · Cohort 1 · Guided Capstone, Supplementary Sitting · SkyWays Logistics is a fictional company and all data is synthetic

Version 1.0

October 2026